

Gelişmiş Ağ Cihazı Yönetimi ve Güvenliği

- Erişim Katmanı Güvenlik Tehditleri
- DHCP Snooping Yapılandırması
- Radius ve Tacacs Kullanarak Harici Kimlik Doğrulama
- SNMP Protokolü ile Ağ Yönetimi
- Switch Yığınlama (Stacking)
- Bulut Bilişim (Cloud Computing) ve Sanallaştırma (Virtualization)
- Software Defined Network (SDN)
- Servis Kalitesi (QoS)

Eriřim Katmanı Gvenlik Tehditleri

Cisco'nun 3 katmanlı hiyerarřik ađ modelinde en altta bulunan Access Layer (Eriřim Katmanı), son kullanıcıların (PC, IP Telefon, Kablosuz Eriřim Noktaları vb.) ađa bađlandığı katmandır. Bu katman aynı zamanda gvenlik tehditlerine en ađık katmandır nk fiziksel eriřim genellikle buradan sađlanır.

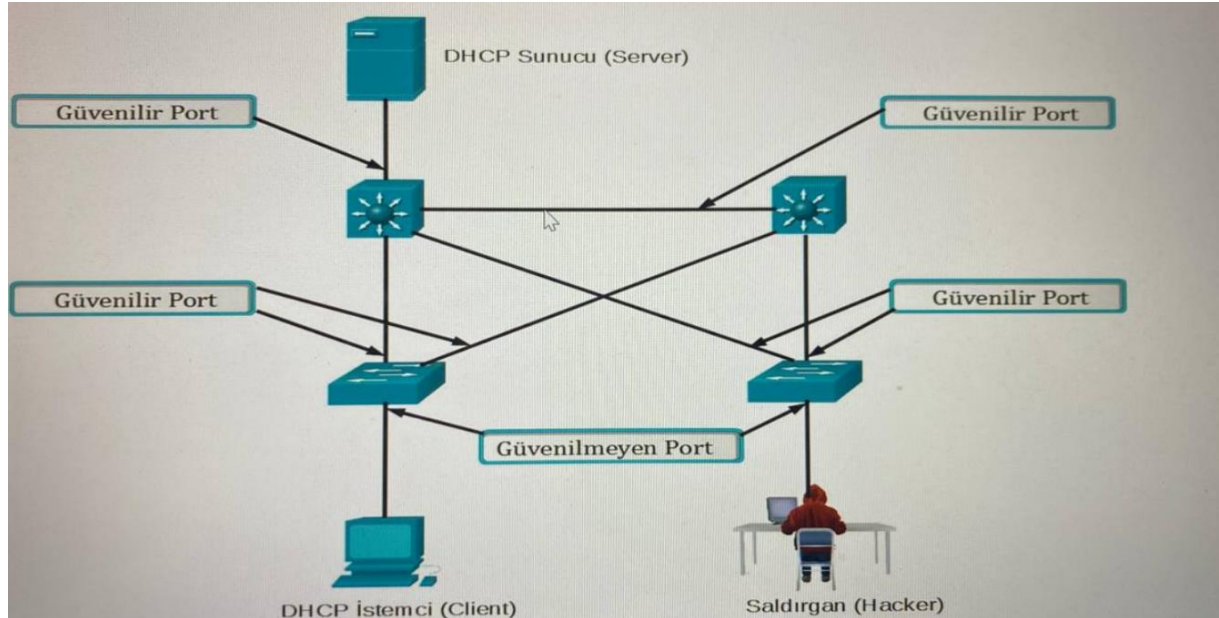
Eriřim katmanında karřılařılabilecek bařlıca Layer 2 saldırı trleri ve bunlara karřı alınan temel gvenlik nlemleri řunlardır:

DHCP Spoofing ve DHCP Snooing (DHCP Mdahesi)

Saldırgan, sahte bir DHCP sunucusu kurarak kendisine DHCP Discover paketleri gnderen istemcilere yanlıř IP, subnet mask, default gateway ve DNS bilgileri dađıtabilir. Bylece Man-in-the-Middle (Ortakdaki Adam) saldırısı yapabilir.

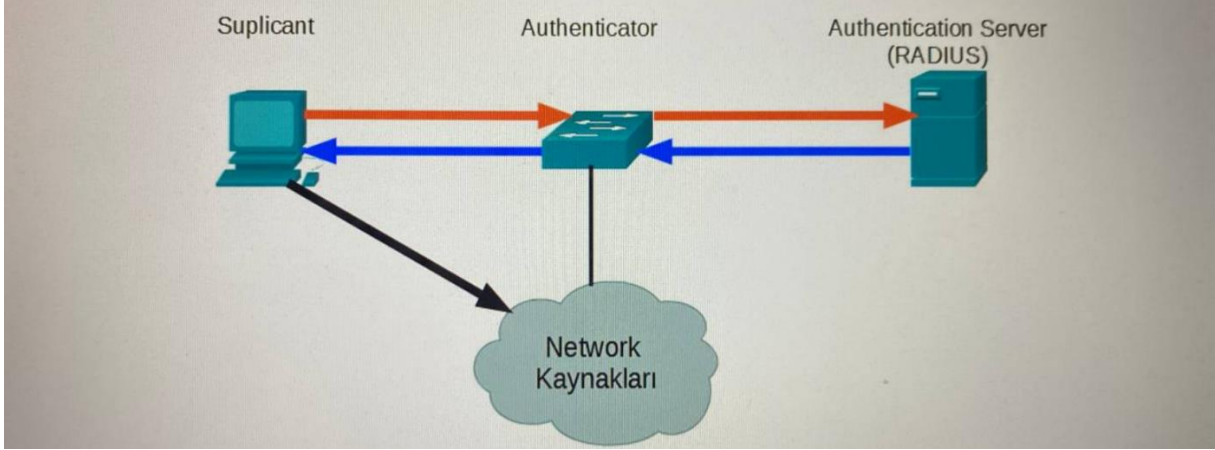
zm: DHCP Snooping

- Switch'te DHCP Snooping etkinleřtirilir.
- Gerek DHCP sunucusunun bađlı olduđu portlar trusted (gvenilir) olarak iřaretlenir.
- Diđer tm portlar otomatik olarak untrusted (gvensiz) olur.
- Untrusted portlardan gelen DHCP Offer, ACK, NAK gibi sunucu mesajları switch tarafından drop edilir.
- Bylece sadece gvenilir porttan gelen DHCP yanıtları istemciye ulařır.



802.1X – Kimlik Tabanlı Ağ Erişimi (Port-Based Authentication)

Port Security sadece MAC adresi kontrol eder. Yetkili bir kullanıcı kendi bilgisayarını çıkarıp başka birinin (örneğin arkadaşının) laptopunu takarsa, Port Security bunu engelleyemez çünkü MAC adresi öğrenilmiş olur.



Çözüm: 802.1X

- Kullanıcı ağa bağlanmadan önce kimlik doğrulaması yapmak zorundadır.

Supplicant (istemci); kullanıcı bilgisayarını. **Authenticator**, Switch (801.1x destekleyen bir switch) ve **Authentication** Server dediği RADIUS sunucusu.

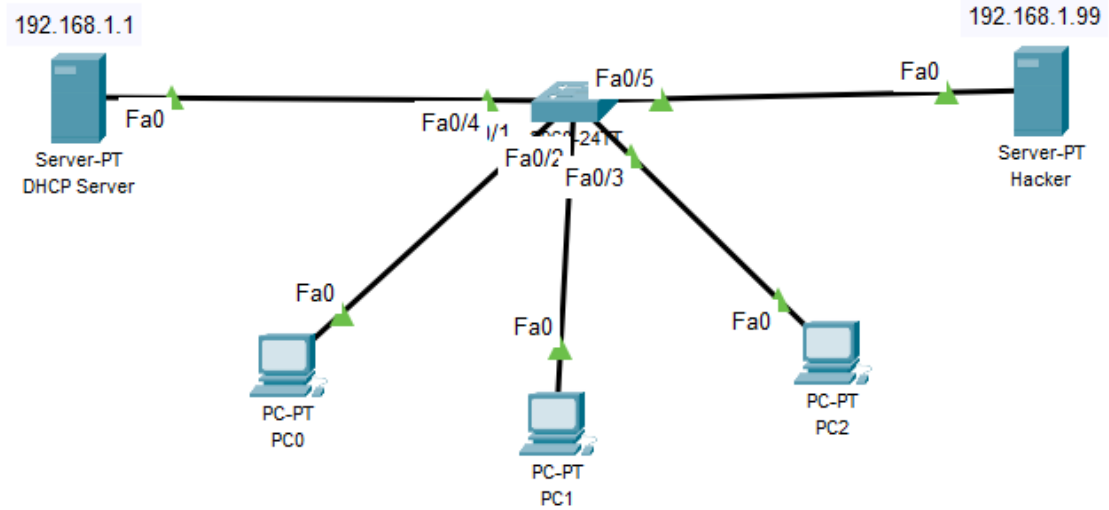
Şu şekilde çalışır:

1. Kullanıcı kabloyu takar → Port başlangıçta **unauthorized** durumdadır.
2. Switch, istemciden kimlik bilgisi ister.
3. İstemci kullanıcı adı/parola gönderir.
4. Switch bu bilgileri RADIUS sunucusuna iletir.
5. RADIUS “Accept” ederse port **authorized** olur.
6. “Reject” ederse port kapalı kalır veya sadece guest VLAN’a yönlendirilir.

Bu sayede aynı porta başka bir cihaz takılsa bile, kimlik doğrulaması yapılmadan ağa erişemez.

DHCP Snooping Yapılandırması

Aşağıdaki yapıda gerçek bir DHCP Sunucusu ve DHCP sunuculuğu yapmaya çalışan bir sahtekar var. Bakalım client'lar hangisine güvenip IP bilgilerini alacak.



Gerçek sunucunun dağıttığı IP havuzu:

SERVICES

HTTP

DHCP

DHCPv6

TFTP

DNS

SYSLOG

AAA

NTP

EMAIL

FTP

IoT

VM Management

Radius EAP

DHCP

Interface: FastEthernet0 Service: ☒ On ☐ Off

Pool Name: serverPool

Default Gateway: 192.168.1.1

DNS Server: 8.8.8.8

Start IP Address: 192 168 1 2

Subnet Mask: 255 255 255 0

Maximum Number of Users: 254

TFTP Server: 0.0.0.0

WLC Address: 0.0.0.0

Add

Save

Remove

Pool Name	Default Gateway	DNS Server	Start IP Address	Subnet Mask	Max User	TFTP Server	WLC Address
serverPool	192.168...	8.8.8.8	192.168...	255.255...	254	0.0.0.0	0.0.0.0

Bu da sahtekarın dağıtmaya çalıştığı IP bilgileri: Default gateway adresi ve DNS adresi olarak kendini vermiş ki trafiği kendisine çeksin.

The screenshot shows a network configuration interface with a sidebar on the left labeled 'SERVICES' containing buttons for HTTP, DHCP (selected), DHCPv6, TFTP, DNS, SYSLOG, AAA, NTP, EMAIL, FTP, IoT, VM Management, and Radius EAP. The main area is titled 'DHCP' and contains the following fields:

- Interface: FastEthernet0
- Service: ☐ On, ☒ Off
- Pool Name: serverPool
- Default Gateway: 192.168.1.99
- DNS Server: 192.168.1.99
- Start IP Address: 192, 168, 1, 2
- Subnet Mask: 255, 255, 255, 0
- Maximum Number of Users: 254
- TFTP Server: 0.0.0.0
- WLC Address: 0.0.0.0

Below the fields are three buttons: 'Add', 'Save' (highlighted), and 'Remove'. At the bottom is a table with the following data:

Pool Name	Default Gateway	DNS Server	Start IP Address	Subnet Mask	Max User	TFTP Server	WLC Address
serverPool	192.168....	192.168....	192.168....	255.255....	254	0.0.0.0	0.0.0.0

Bu sayede bilgisayarlar IP bilgilerini sahtekar tarafından edinmiş olabilir.

Not: Uygulamada eğer tekrar IP request atmasını istiyorsan terminalden **ipconfig /renew** diyebilirsin.

Bizim yapmamız gereken şey DHCP Snooping yani DHCP Müdahalesi. Bu sayede switchin sadece gerçek DHCP Server'a giden port'unu trusted olarak ayarlayacağız, otomatik olarak zaten diğerleri untrusted duruma geçiyor.

```
Switch(config)#
Switch(config)#ip dhcp snooping vlan 1
Switch(config)#int fa0/4
Switch(config-if)#ip dhcp snooping trust
Switch(config-if)#end
Switch#
%SYS-5-CONFIG_I: Configured from console by console
Switch#
```

Bizde vlan olmadığı için varsayılan vlan 1 'e atıyoruz.

Radius ve Tacacs Kullanarak Harici Kimlik Doğrulama

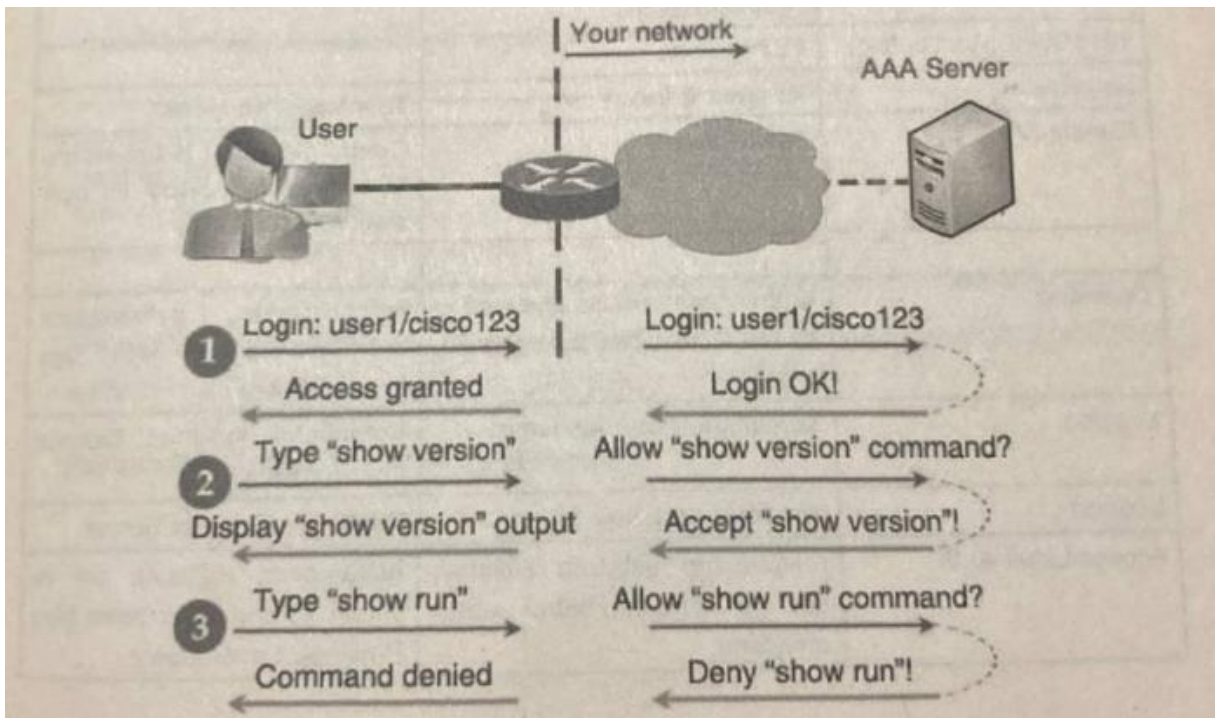
AAA Authentication

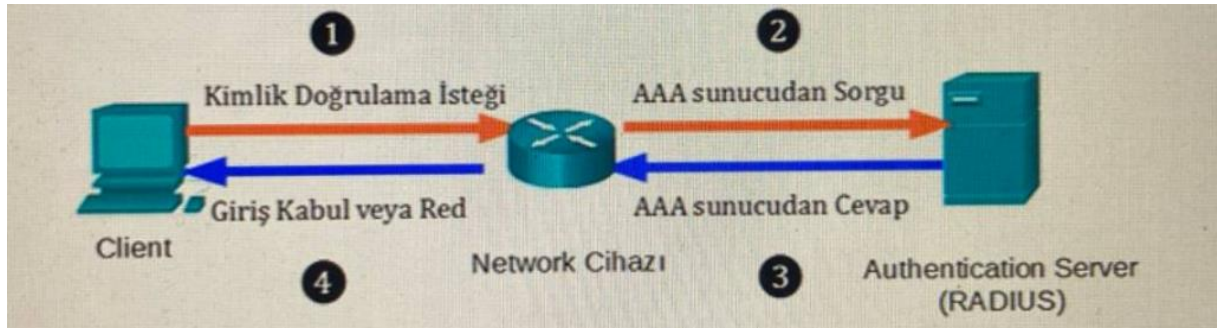
Eğer ağıınızda çok fazla sayıda yönetilebilir cihazınız var ise bunları merkezi olarak yönetmeniz akıllıca olur. Çünkü işe yeni başlayan bir Network yöneticisi için tüm cihazlarda kullanıcı ve yetki tanımlaması yapmak zorunda kalırsınız. Ayrıca birden fazla Network yöneticisinin bulunduğu ortamda olası konfigürasyondan dolayı ortaya çıkan sıkıntılardan doğal olarak kimse sorumluluğu da üstüne almaz. Bunun dışında default şifresinde kalan ya da güvensiz giriş şifresi olan cihazlar brute-force (şifre deneme) ataklarına karşı zayıf durumdadır. Tüm bu ihtiyaçlar merkezi bir yönetimin, yetkilendirmenin ve merkezi konfigürasyon değişikliği kayıtlarını tutmayı gerektirir. Tüm bu ihtiyaçların çözümü için geliştirilen sistemler AAA başlığı altında toplanır.

Kısaca AAA ifadesi:

- Authentication: Kullanıcı adı ve şifre doğrulaması / Kullanıcı adım arda / şifre: cisco / tamam doğru devam et.
- Authorization: Kullanıcı neler yapabilir ve nerelere ulaşabilir? / Arda kullanıcısı CCNA sunucusuna http ve HTTPS portlarından erişebilir.
- Accounting: Kullanıcı ne kadar sürede ne zaman neler yaptı? Arda kullanıcısı 2 saat boyunca erişim sağladı. Gi 0/1 interface'inin IP'sini değiştirdi.

AAA devre alındığında bir cihaza erişim adımları şöyledir:





AAA Sisteminin avantajları:

- Güvenli giriş: Kullanıcılar direkt olarak AAA sunucuyu görmez. Cihazlar kullanır.
- Kolay yönetim: Merkezden yönetilen sistemler için idealdir.
- Firewall ve diğer güvenlik cihazları da AAA sunucusunu kullanabilir.
- Özel komutları komut bazında kabul ya da reddedebilme.
- Her komutun kaydı kullanıcı bilgisi ile birlikte tutulabilmekte ve sonradan kolayca analiz edilebilmektedir.

AAA Sisteminin dezavantajları:

- Tüm gelen sorgulamaları saliseler içinde kontrol edip onaylayabilecek kadar güçlü bir sunucu ve ağ cihazlarında bu sunucuya doğru hızlı bir erişim gerekebilir.

AAA sistemlerinde genel olarak 2 protokol kullanılır:

- RADIUS (Remote Authentication Dial In User Service)
- TACACS+ (Terminal Access Controller Access-Control System)

Her iki sistemin karşılaştırılması:

	RADIUS	TACACS+
Transportation & Ports	UDP port 1812/1645 (Authentication)	
Encryption Standards	Yalnızca Şifreleri Açık Standart	Tüm başlık ve paketi Cisco Kökenli (Günümüzde Cisco'nun da desteği ile açık standart RFC1492)
Operation	Authentication and authorization bir bütünlük içinde kontrol edilir, onaylanır.	Authentication, authorizatin and accounting ayrı ayrı kontrol edilir.
Logging	Komutların Logu tutulmaz	Komutların kullanıcı bazında logu tutulabilir

Support	Windows ve Linux Server	Windows ve Linux Server
Access-Level Auth	Kullanıcının yalnızca kullanıcı adı ve şifresi kontrol edilip onaylanır.	Kullanıcının kullanıcı adı ve şifresi kontrol edilir buna göre Privilege Level atanır.

Görüldüğü gibi her iki yönteme de bakıldığında RADIUS sisteminin TACACS'a göre bir avantajı bulunmamaktadır. RADIUS çok eski ve aslında basic olarak Authentication yapsın yeter anlayışıyla ortaya çıkmıştır. Radius sistemlerde Authentication ve Authorization birlikte gönderilir. Bu da Tacacs sistemlerine göre yavaş çalışması demektir.

Genel olarak AAA Yapılandırılmasında:

1. Enable "new model" AAA
2. TACACS+ ya da RADIUS sunucuyu tanımla
3. Authentication ve Authorization metodunu belirle
4. Tanımlanan güvenlik protokolünü VTY'nin içinde uygula

Basit bir AAA uygulamasındaki komutlar:

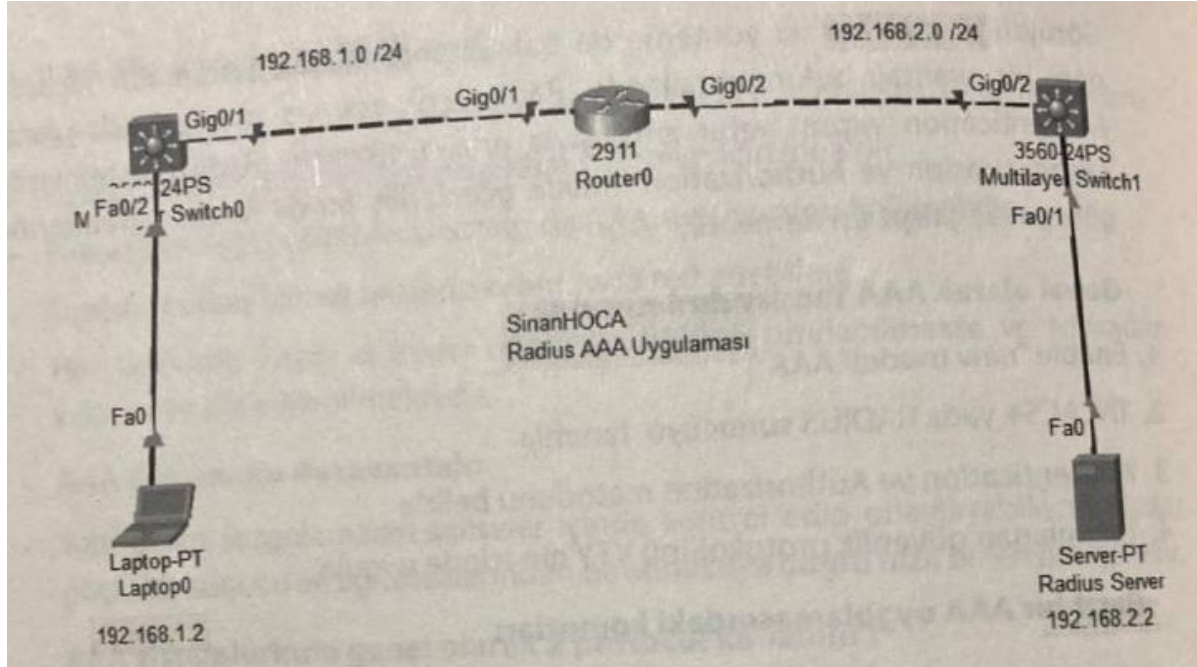
```
Switch(config)#username test2 secret Pa55w0rd
Switch(config)#aaa new-model
Switch(config)#radius-server host 192.168.1.2 key MySecretP@ssword
Switch(config)#aaa authentication login MY_AUTHEN_GROUP group radius
local
Switch(config)#line vty 0 4
Switch(config)# login authentication MY_AUTHEN_GROUP
```

Basit bir TACACS+ konfigürasyonu:

```
aaa new-model
aaa authentication login default group tacacs+ local
tacacs-server host 10.10.10.1
tacacs-server key login@pass!
```

Buradaki tacacs-server key cihazın tacacs sunucusuna erişemediği durumlarda kullanılması içindir. Buna Tacacs OFF şifresi de denir.

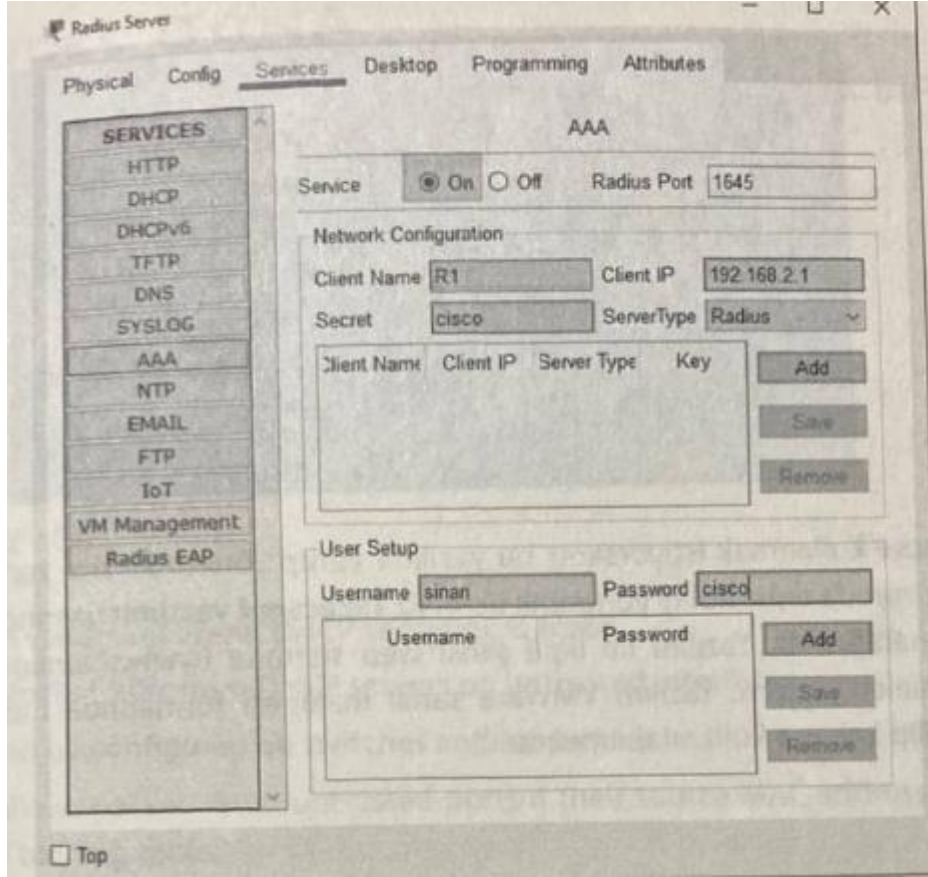
Packet Tracer Radius AAA Uygulaması:



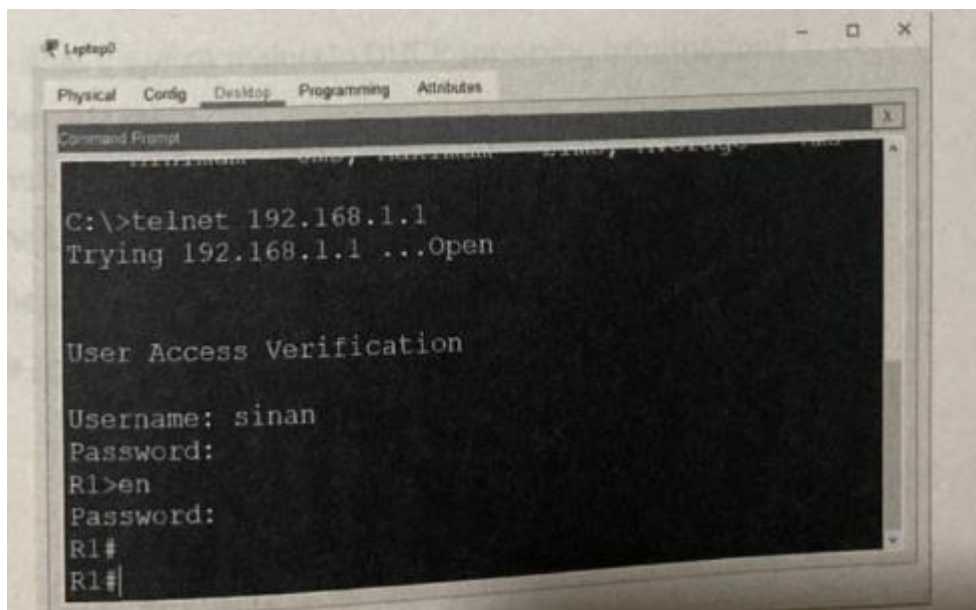
Özellikle sunucuya ve client'a IP verelim ve router'ın interface'lerinin ayarlamalarını yapalım. **Özellikle hostname zorunludur.** Sonrasında:

```
R1(config)#aaa new-model
R1(config)#radius-server host 192.168.2.2 key cisco
R1(config)#aaa authentication login default group radius (gerçekte
local da var)
R1(config)#enable pass cisco
R1(config)#line vty 0 4
R1(config-line)#login authentication default
R1(config-line)#exit
```

Şimdi de sunucumuzdaki AAA Yapılandırmasını yapalım:



Aşağıdaki kısımda hem sinan isimli hem de ciscoR1 isimli, şifreleri cisco olan 2 kullanıcı oluşturulmuştur. Şimdi de client dan telnet deneyelim:



```
C:\>telnet 192.168.1.1
Trying 192.168.1.1 ...Open

User Access Verification

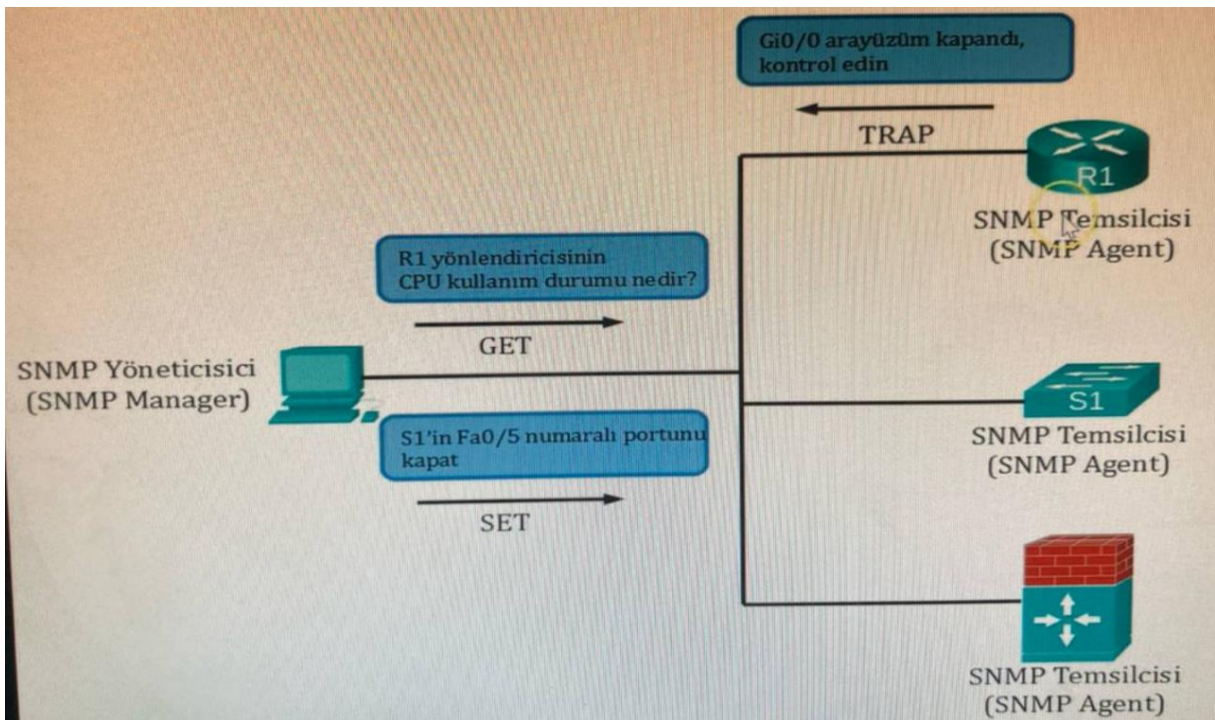
Username: ciscoR1#
Password:
R1>
```

Tacacs+ kullanmak istiyorsanız bu yazılımı kurup yönetmek zor gelebilir. Bu durumda daha kolay yönetimli ücretsiz **Tacacsgui yazılımı** üzerinden **Tacacs** kullanabilirsiniz.

SNMP Protokolü ile Ağ Yönetimi

Ağların büyümesiyle birlikte cihazların tek tek yönetilmesi imkansız hale gelir. Bu noktada devreye, merkezi bir izleme ve yönetim protokolü olarak **SNMP (Simple Network Management Protocol)** girer. Bugün router, switch, güvenlik duvarı, hatta yazıcı gibi ağ cihazlarının durumunu uzaktan takip etmenin en pratik yolu SNMP'dir.

- Uygulama katmanında (Layer 7) çalışır.
- Ağ cihazlarını izleme (monitoring) ve yönetme (configuration) amacı taşır.
- İletişim için çoğunlukla UDP kullanır.
- Manager tarafından cihazlara istek atılır, cihaz Agent modülü üzerinden cevap döner.



SNMP Manager, İzleme/Yönetim için kullanılan sistemdir. Bir sunucuda veya PC'de Windows veya Linux üstüne kurulan ücretli veya ücretsiz bir yazılım olabilir. Agent'lara GET, SET gibi paketler yollar, Trap mesajlarını da alır.

SNMP Agent, router ve switch gibi ağ cihazlarının içinde çalışan temsilci modüldür. Manager'dan gelen taleplere yanıt verir. Cihazın iç işleyiş bilgilerini toplar ve MIB'e kaydeder.

MIB (Management Information Base), SNMP Agent'ın cihaz hakkında bilgi tuttuğu yönetim bilgi veri tabanıdır. Bu bilgiler OID (Object ID) ile tanımlanır. Örneğin, CPU kullanımı, port durumu (up/down), trafik istatistikleri, hata logları vb.

NOT: Manager'dan Agent'a doğru Sorgu/Ayar mesajları UDP 161 Portunu kullanırken, tersi yani Agent'dan Manager'a doğru Trap/Uyarı mesajları UDP 162 Portunu kullanır.

SNMP Versiyonları

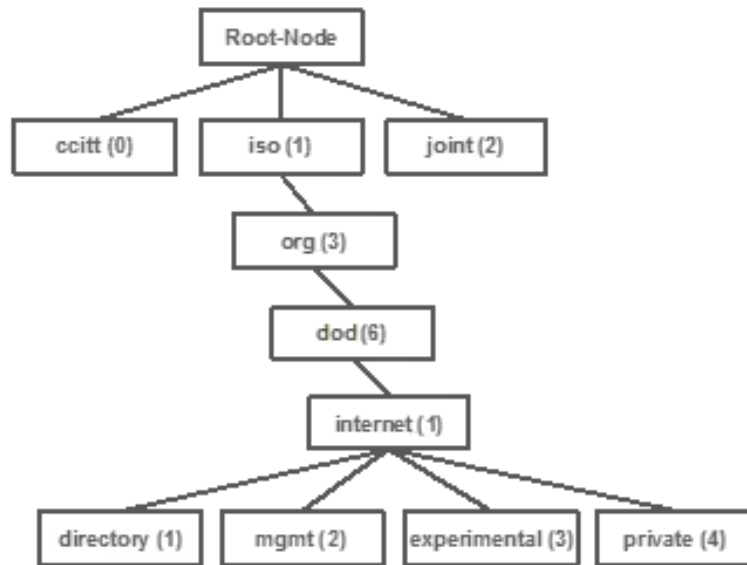
SNMP'nin 3 ana versiyonu vardır:

Sürüm	Güvenlik Modeli	Kimlik Doğrulama	Şifreleme
SNMPv1	noAuthNoPriv	Community String	Hayır
SNMPv2c	noAuthNoPriv	Community String	Hayır
SNMPv3	noAuthNoPriv	Kullanıcı Adı	Hayır
SNMPv3	authNoPriv	MD5 veya SHA	Hayır
SNMPv3	authPriv	MD5 veya SHA	DES veya AES

- V1 ve v2c'de Community String şifrelenmeden (plain text) gönderilir. Bu güvenlik açığıdır.
- V3 ise authPriv veya authNoPriv gibi güvenlik modelleri ile hem kimlik doğrulama hem de şifreleme sunar. Bu yüzden modern ağlarda SNMP kullanımı için önerilen SNMPv3'tür.
- V1 ve v2c'deki Community String modları iki çeşit. Read-only (ro), sadece veri okur, ayar değişikliği yapamaz. Read-Write (rw), okur ve yazar (configuration change yapabilir.). Güvenlik gereği Community String her ne kadar password gibi davranırsa da şifreli taşınmadığı için sadece ACL ile izin verilen Manager IP'leri erişmelidir.

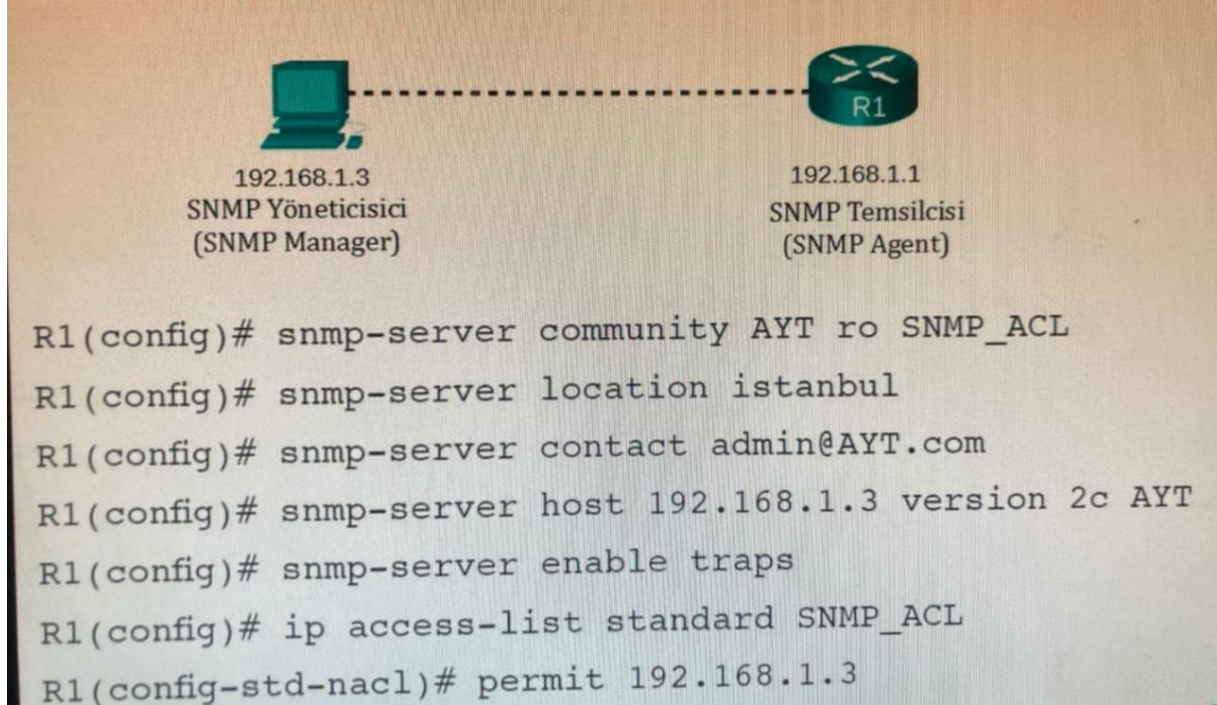
OID Ağaç Yapısı

MIB içindeki her bilgi veya nesne bir OID ile ifade edilir. Cisco cihazlarında OID'ler evrensel bir ağaç yapısına göre numaralandırılır.



Bu yapı giderek özelleşir ve Manager'a dönen cevaplarda OID numaraları uzun/karmaşık olabileceği için anlamlandırmak adına SNMP Object Navigator gibi araçlar veya yazılımlar kullanılır.

SNMP Yapılandırması



- **AYT** adında bir Community String tanımlandı.
- **“ro”** ile read-only izin verildi.
- **SNMP_ACL** access list'i oluşturulup sadece **192.168.1.3** yani Manager IP'sine erişim izni verildi. UDP üzerinden gittiği için UDP açıkları olan bir protokol bu yüzden ACL uygulanmalı.
- **“enable traps”** ile cihazdan Manager'a uyarı mesajları (ör. Interface kapanması) açık hale getirildi.

Doğrulamak için **de show snmp community** gibi komutlar kullanılabilir.

Switch Yığınlama (Stacking)



Switch stacking, birden fazla fiziksel switch'in, özel "stack" portları ve kabloları kullanılarak birbirine bağlanıp tek bir mantıksal switch gibi çalıştırılmasıdır. Örneğin aynı seriden 4 adet switch'i stack'e alıp sanki tek bir büyük switch varmış gibi yönetebiliriz. Bu yapılandırmaya **switch yığınlama (stacking)** denir.

Switch üzerinde normal Ethernet portlarından farklı stack portları bulunur. Bu portlar özel stack kabloları ile birbirine bağlanır. Genelde kablo uzunluğu maksimum 3 metre civarındadır, yani switch'lerin aynı rack/kabin içinde olması beklenir.

Avantajları:

Yönetim Kolaylığı

- Her switch'e ayrı ayrı konsol kablosu takmak zorunda değiliz.
- Bir tane master switch seçilir ve yönetim onun üzerinden yapılır.
- Yapılandırma (running-config, startup-config) master üzerinde tutulur.

Tek IP ile Yönetim

- Stack, ağda tek bir cihaz gibi görünür.
- Yönetim için bir tane management IP verirsın.
- O IP'ye bağlandığında aslında master switch'e ulaşırsın, ama config tüm stack'e yayılır.

Yedeklilik

- Her switch, master olmaya adaydır.
- Master switch bozulursa veya erişilmez olursa priority değeri yüksek olan switch master olur.

Layer 2 / Layer 3 Protokolleri Açısından Stack

- Tüm stack, L2/L3 protoklleri açısından tek bir switch gibi görünür.
- STP açısından, stack içindeki tüm switchler tek bir bridge ID ile temsil edilir.
- Yani sanki tek bir köprü (bridge) varmış gibi davranır.
- Benzer şekilde diğer protokoller (CDP, VTP, DTP vs.) de stack'i tek bir cihaz olarak algılar.

Not: Eğer bir switch'in power'ı bozulursa, diğer switch'ler tarafından gücü almış olur.

Bulut Biliřim (Cloud Computing) ve Sanallařtırma

Cloud computing, her řeyin bulut saęlayıcıları veya řirketinizdeki iç bulut sunucularından sunulduęu bir yaklařımdır. Cloud yaklařımlarının tümünde ařaęıdaki avantajlar günümüz Türkiye'si için mevcuttur.

- Kolay güncelleme seęenekleri
- Her yerden erişebilirlik, kolay yönetim
- Cloud sistemlerinin ödemeleri aylık ya da yıllık periyodik olarak ödendięinden vergiden düşebilme
- Daha esnek Disaster Recovery (Felaket Senaryoları, Coęrafi Yedeklilik)

Cloud Computing konusuna girmeden önce adım adım önce sunucular, sonra sanallařtırma ve sonra da Data Center yapılarındaki kullanım ve uygulamalarına bakalım.

Sunucular ve İş İstasyonları (Workstations)

Önce sunucu kavramından başlayalım. Sunucular **7/24** ve **365 gün** çalışacak şekilde özel olarak üretilmiř güç kaynaęı, anakart, RAM, işlemci ve disklerden oluşur. Birçok sunucuda birden fazla ethernet ve güç kaynaęı, birden fazla disk bulunur.

Sunucularda her řey **kesintisiz verimlilik** üzerinde kurgulanmıřtır. Sunucular genelde yüksek RAM ve performanslı işlemciler barındırır. Sunuculara da işletim sistemi kurulmadan disklerin yedeklilięi için **RAID** yapılır. Bare Metal sunucuları, üzerinde Windows ya da Linux gibi tek bir işletim sistemi barındıran fiziksel sunuculardır. Genelde işlemci ve Ram'leri fazla olmaz. Sunucuları yapı olarak **iki** çeřitir.

- 1- **Rack Sunucular:** Yatay olarak konumlandırılır ve kabinette üst üste yerleřime uygundurlar. Genelde 2U yükseklięindedirler ama **1U** ve **4U** çeřitleri de bulunabilmektedir. Çok sayıda sunucu ihtiyacı olan yerler için daha az yer kapladığından ve düzeninden dolayı tercih edilir. Genelde Data Center'ların içi bu sunucular ile doludur. Birçok sunucunun olduęu bir yerde en önemli 3 unsur da **soęutma**, **kesintisiz aę iletişimi**, **kesintisiz elektrik enerjisidir**. Genel olarak soęutma, sunucularda %30'a kadar performans artışı da saęlar.



- 2- **Tower Sunucular:** Dik olarak konumlandırılır ve genelde evde kullandığımız normal masaüstü bilgisayarların boyutunda ya da daha büyüktür.



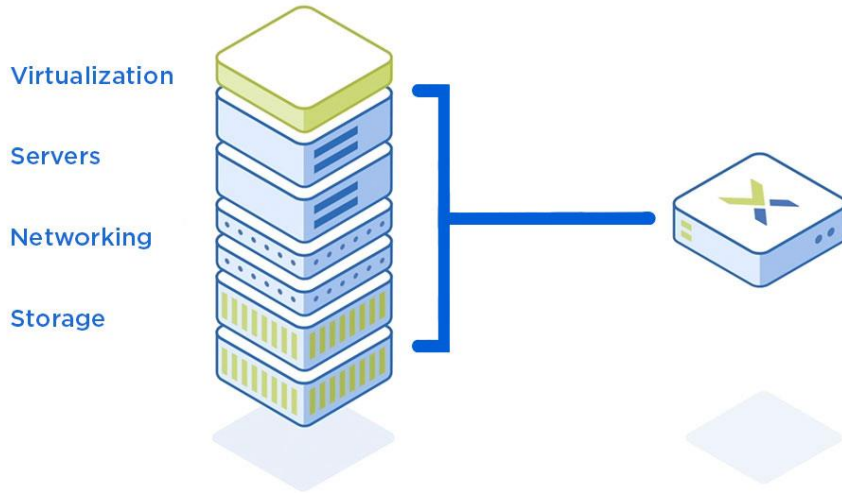
Bir sunucuda tek bir işletim sistemi olsa bile üzerinde bir sürü **uygulama/role/görev** olabilir. Sık kullanılan bazı sunucu görevleri ise:

- **Active Directory Service**
- **DNS Server**
- **E-mail Server**
- **Web Server**
- **File Server**
- **Remote Desktop Server**

Şüphesiz büyük kurum ve kuruluşlarda her bir görev için ayrı sunucu kullanılır. Böylece olası sorunlarda yalnızca o sunucudaki görev aksamış olur. Genelde sunucuların ekran kartları yalnızca ekrana görüntünün iletilmesini sağlayacak **minimum performanstadır**. Özellikle 3 boyutlu görüntü üreten, üzerinde çalışan kişiler ya da yüksek kalitede videolar üzerinde çalışan kişilerin ise hem yüksek işlemci ve RAM gücüne hem de yüksek ekran kartı performansına ihtiyacı olur. Sunucularda yüksek işlemci, disk ve RAM gücü olmakla birlikte yüksek performanslı ekran kartları yoktur. Bu ihtiyaç **iş istasyonları** ile giderilmiştir. İş istasyonları, yüksek performanslı sunuculara yine **yüksek performanslı ekran kartlarının** da eklenmesiyle oluşturulur. Genelde de sunucu değil **client** işletim sistemi yazılımı (Windows 10 gibi) üzerinde çalışırlar. İş istasyonlarının genelde masaüstü (tower) ya da dizüstü modelleri bulunmaktadır.

Bizler bir sunucu alacağımız zaman önce ihtiyacımız olan **performansa** karar veririz. Bu sunucuyu ne için kullanacağız ve bize bu iş için ne düzeyde işlemci, RAM, disk içeren sunucu gerekiyor? Peki, ya ihtiyaçlarımız artarsa? O zaman bu sunucunun performansını artırmak için işlemci, RAM, disk ekleme imkanlarımız var mı? İşte tüm bu kritik sorular ve kolay yönetim bizleri **sanallaştırmaya** yöneltmiştir. Sanallaştırma sistemlerinde yüksek performanslı sunucuları ve kaynakları sanal sunucular kullanmakta ve duruma göre bir sanal sunucunun ihtiyacı olan işlemci, RAM, disk gibi değişkenler kolayca artırılabilir. Özellikle disk yönetimi çok değişkenlik gösterdiğinden ve çok çabuk dolabildiğinden genelde sanallaştırmalarda harici bir **storage ünitesi** kullanılır. Storage üniteleri birçok diskli yine gelişmiş raid teknolojileri ile adeta birleştirip sanal sunuculara sunmaktadır. Storage üniteleri ile sunucular arasında ise **min 10Gbps**'lik yüksek hızda **fiber kablolar** kullanılır. Böylece fiziksel sunucuların içindeki sanal sunucular storage'deki kendilerine tanımlanan disklerle sanki kendi fiziksel sunucularının içindeymiş gibi hızlı ulaşabilirler.

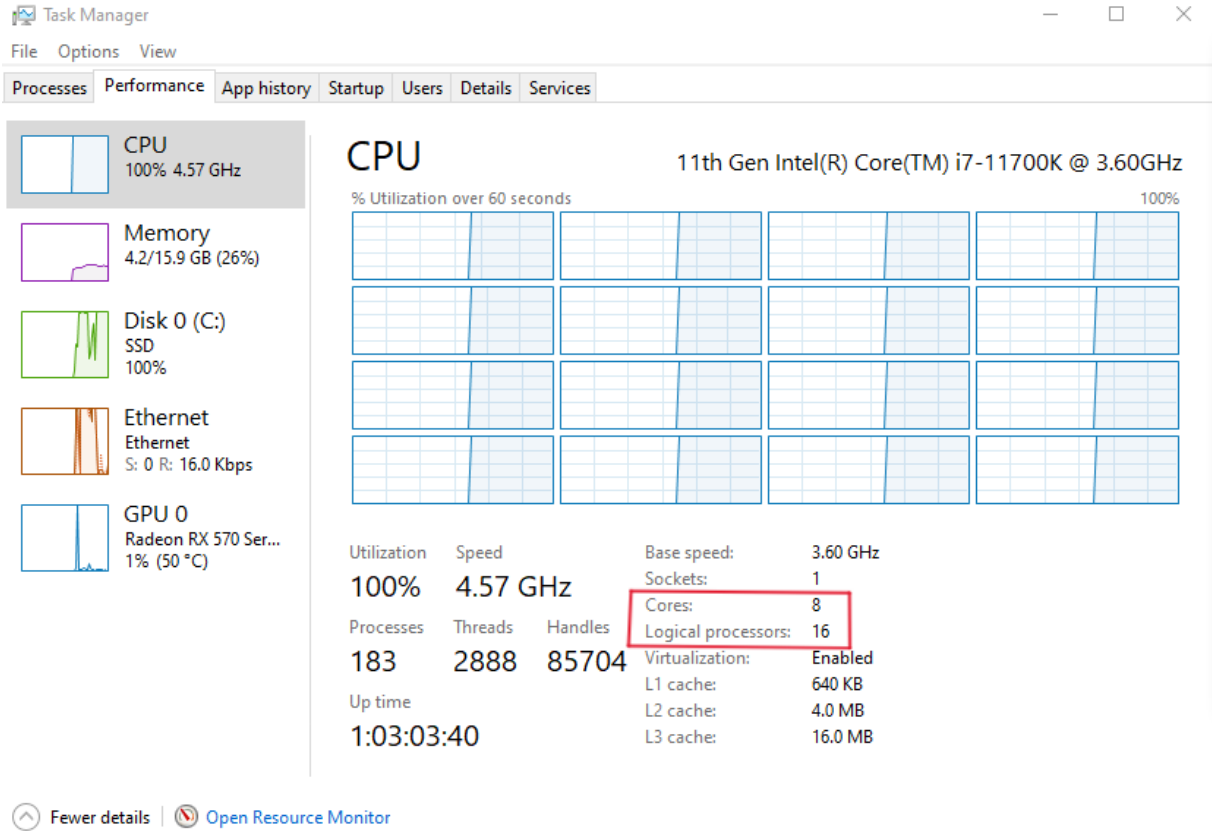
Genel olarak bir sanallaştırma projesinde ihtiyacımız olan fiziksel sunucular, storage üniteleri ve bunlar arasındaki yüksek hızda bağlantılar, üreticileri tüm bunların bir arada olduğu sistemleri de üretmeye yöneltmiştir. Bu tür hepsi bir arada sistemlere de **Hyper-Converged** sistemler denir.



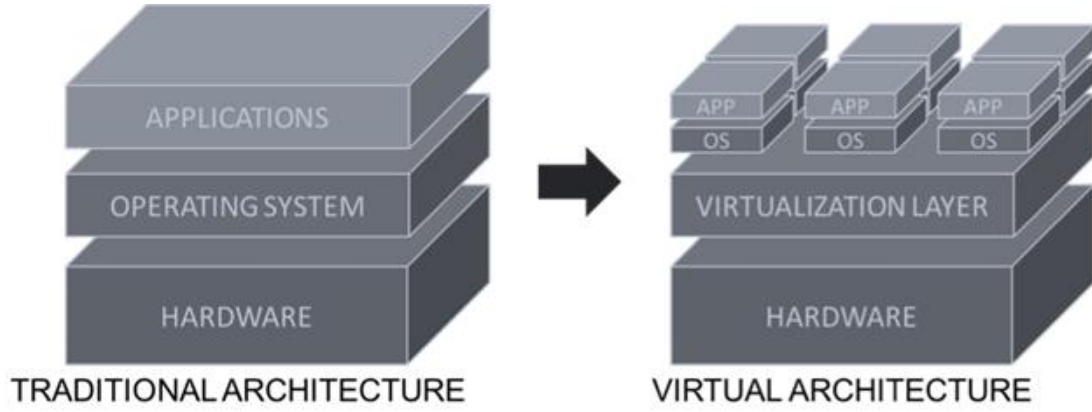
Hyper-Converged bir yapıda yönetim; sorun bulma ve giderme, sanallaştırma yazılımlarıyla uygunluk, maliyet, enerji tüketiminden tasarruf, genel sahip olma ve işletme maliyetlerindeki uygunluk gibi birçok **avantaj** vardır.

Server Virtualization

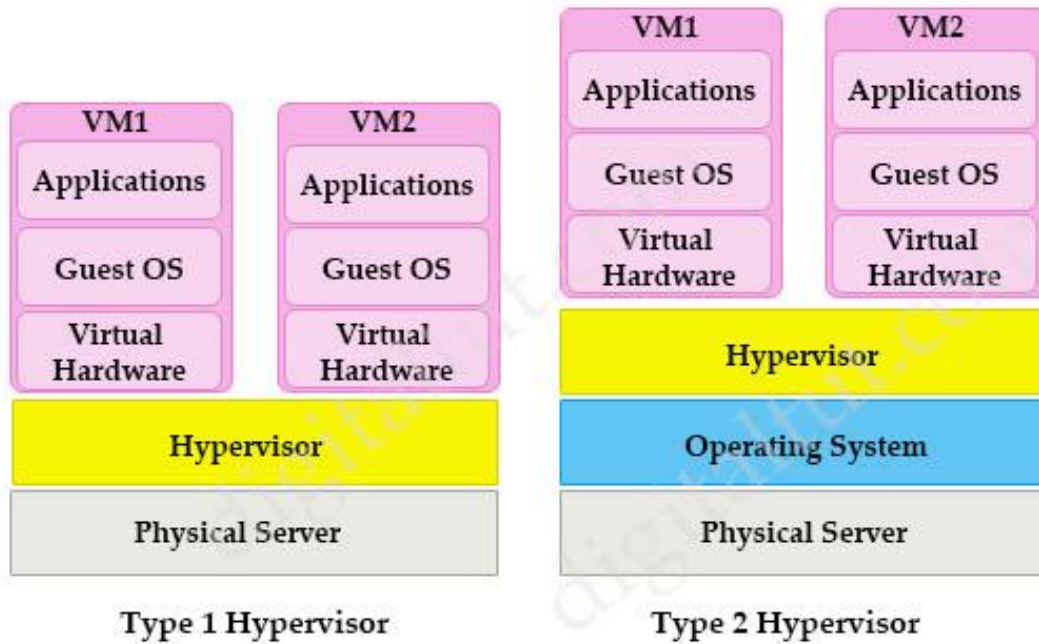
Son 10 yılda işlemcilerdeki core sayısı inanılmaz şekilde arttı. Önce tek core'lu işlemcilerden çift çekirdekli'lere sonrasında ise dört çekirdek günümüzde ise 32 çekirdeğe kadar barındıran işlemciler bulunmaktadır. Ayrıca her bir fiziksel core, **sanal iki core** (thread) şeklinde de çalışabilmekte ve **tek bir core, farklı iki işlemi yürütebilmektedir**. En basitinden aşağıdaki bir i7 işlemcinin core ve thread sayılarına bakabilirsiniz:



Üstteki resimde de gördüğünüz gibi sistem işlemcideki 8 fiziksel core'u **16 sanal core** gibi çalıştırmaktadır. Aynı hızlı gelişme RAM tarafında da yaşandı. Günümüz client sistemleri bile artık çok ciddi RAM kapasitelerini desteklemektedir. Bu teknoloji gelişmeleri sanallaştırmayı daha da kolaylaştırdı. Böylece çok güçlü işlemci ve RAM'e sahip sistemler kurup **sanallaştırarak**, sanal sunuculara esnek bir şekilde fiziksel kapasite paylaştırılabilmektedir. Ayrıca sanallaştırma sayesinde sadece belli zamanlarda daha yüksek performansa ihtiyacı olan sunucuların (örneğin kampanya dönemindeki bir e-ticaret sitesi, ya da sonuç açıklanan bir okul/üniversite web sitesi vb.) bu yoğun zamanda fiziksel kapasitede boşta olan işlemci, RAM gibi kaynakların **geçici kullanılması da** mümkün hale geldi. Normalde sanallaştırma olmayan bir sunucuya bir uygulama kurduğunuzda bu sistemin **katman yapısı** şöyle olur:



Bu tür sistemlerde elbette donanım (hardware) ya da işletim sistemi (OS) katmanında bir sorun oluştuğunda bunların üstünde çalışan tüm uygulamalar olumsuz etkilenecektir. Sanallaştırma günümüzde iki türlü uygulanmaktadır. Bir tanesi mevcut sunucunuzdaki işletim sisteminin üzerine sanallaştırma yazılımı kurup bu sanallaştırma platformu içinde sanalları oluşturmaktır (VMWare Workstation, Hyper-V, VirtualBox vb.). Bu tür sanallaştırma aşağıdaki şekilde Type 2 olarak gösterilmiştir.

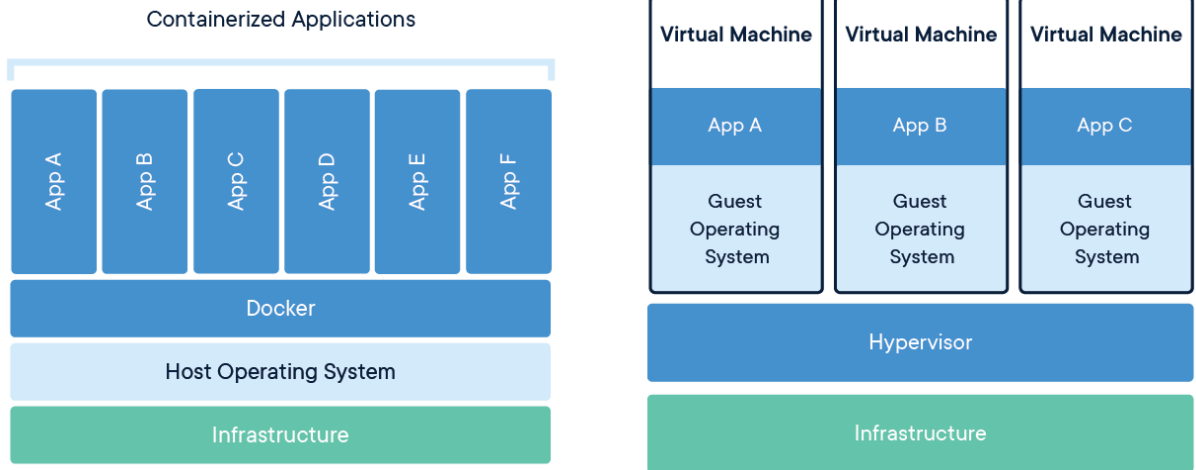


Sanallaştırmanın olduğu katmanda **Hypervisor** katmanı diyoruz. Mevcut sunucu işletim sisteminiz üzerine kurulan bu **sanallaştırma katmanı üzerinde tüm sanalları** taşımaktadır. Lab ortamında sanal makineler kurup denerseniz bir sorun olmadığı halde mevcut sisteminizdeki fiziksel kaynaklarınızın bir kısmını işletim sistemi kullanmakta ve sanal sistemlerinize **daha az kaynak** kaldığını fark edebilirsiniz. Bir sanallaştırma sisteminde biz kaynakların olabildiğince fazlasını sanal sistemlerimizin kullanmasını isteriz. Bu durumda üreticiler yalnızca sanallaştırmaya hizmet eden **light işletim sistemini de içeren sanallaştırma çözümleri** üretmişlerdir. Microsoft'un Hyper-V server, VMWare'in ESXi sistemleri bunlara örnek verilebilir.

Bu tür sistemlerin topolojisi de resimde **Type 1** olarak gösterilmiştir. Bu tür sistemler light bir hypervisor katmanına sahip olduğundan fiziksel kaynakların hemen hemen hepsi sanal sistemlere kalmaktadır. Ayrıca bu tür sistemlerde **sorun yaşama ihtimaliniz de çok daha düşüktür**. Bu tür sistemlerin belki de tek dezavantajı, sistem üzerine bir monitör bağlayarak **görüp yönetemezsiniz**. Sanal sistemlerin yönetimi için farklı bir client makineden bağlanmanız gerekir.

Containers (Docker)

Containers ya da daha sık duyduğunuz adıyla Docker sistemler aslında **light-weight virtual machines** (hafif sanal makineler) olarak bilinse de aslında **sanal bir makine değildir**. Çünkü bu Container'ların ayrı bir interface, kaynak yönetimi genelde yoktur. Docker sistemlerini şöyle düşünebilirsiniz; Bir uygulamayı çalıştırmak için bir işletim sistemine ihtiyacımız var. Bu işletim sistemi üstüne **uygulamamızı kurar ve çalıştırırız**. Fakat biz bu sistemi yalnızca bu uygulama için kurmuşsak ve kullanıyor isek doğal olarak aslında o fiziksel makinedeki tüm kaynakların, tüm performansın o uygulama için kullanılmasını isteriz. Örneklerle ifade etmek gerekirse bir web sunucusu ihtiyacınız var ise önce bir işletim sistemi, üstünde IIS benzeri bir web host yazılımı kurmanız ve ayarlamanız gerekir. Bu durumda işletim sistemi hem ciddi bir kaynak tüketecek, hem de aslında bizim ya da uygulamanın ihtiyacı olmayan bir çok default uygulama arka planda çalışıp kaynak tüketecektir. **Container** ya da **Docker** sistemler gereken uygulamanın gerektirdiği yazılımları da içerecek şekilde yapılandırılan **özel paketlerdir** diyebiliriz.



Docker ya da diğer adıyla Container'ların **avantajlarını** şöyle sıralayabiliriz:

Small: Container imajları sadece ihtiyaç olan yazılımları içerdiğinden boyutları inanılmaz ölçüde küçüktürler. Bu da onlara hem daha az yer hem daha kolay ve hızlı RAM'de çalışabilme yani dolayısıyla performans kazandırır. Örneğin FreeRadius bir Docker içinde FreeRadius için gereken tüm yazılımlar dahil 5MB kadar küçük olabilmektedir.

Portability: Container imajları esnek kullanıma sahiptir. Kendi client makinenizde testlerini yapar sonra isterseniz bu container'ı sunucuda da rahatlıkla çalıştırabilirsiniz. Zaten container'lar uygulamanın çalışması için gerekli olan tüm ek yazılımları da içerdiklerinden çok hızlıdır.

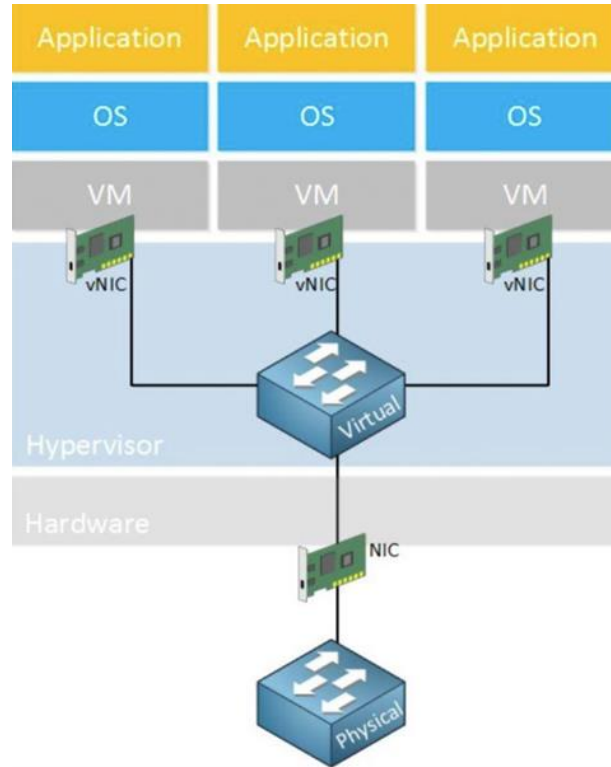
Fast: Container'lar hızlıdır. Bir sanal makineye bağlı olmadıklarından, kendi çalışmaları için gerekli yazılımları da içerdiklerinden çok hızlıdır.

Isolation: Container'lar diğer Container'lardan bağımsız olarak çalışır. Bir Container çöktüğünde sadece kendisi etkilenir, diğerlerini etkilemez.

Günümüzdeki Container tabanlı sistemler; **Docker**, **rkt** (Rocket), **LXD**, **Linux Vserver** ve **Windows Containers**'dir.

Virtual Networking

Günümüzdeki sunucularda giriş model cihazlarda en az 2, normal sunucularda ise en az 4 adet ethernet port bulunur. Sunucu üzerine kurulan işletim sistemi bu ethernetler sayesinde ağa bağlanır. Peki neden bu kadar çok sayıda?



Yukarıdaki şekilde fiziksel sunucu üzerindeki sanal sunucular **tek bir ethernet bağlantısı ile switch'e bağlanır**. Bu tür durumlar ister istemez darboğaza sebep olabilir. Çünkü sunucudaki tüm sanal cihazlar ve fiziksel sunucunun kendisi tek bir ağ bağlantısı üzerinden haberleşmektedir. Eğer bu bağlantı 1 Gbps gibi aslında normal bir bağlantı hızına sahip olsa da bu bağlantıyı birçok cihaz aynı anda kullanmaya çalışınca ağ erişiminde **yavaşlamalar** olacaktır. Bu yüzden aslında sanallaştırma yapılan sunuculara kendi üzerlerinde yoksa **HBA kart** denilen, 10 Gbps fiber **bağlantıyı destekleyen** kartlar takılır. Bir öneridir ki eğer hiçbir şey yapamıyorsanız

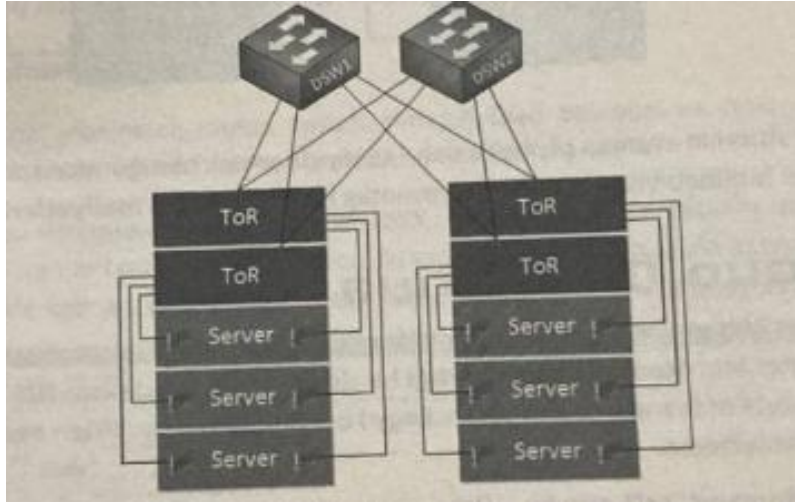
en azından farklı sanal sunucuları farklı ethernetlere yönlendirirseniz ağ performansınız artar. İstenirse sunucular farklı vlan tag'leri ile **farklı ağlara** da bağlanabilir.

Tüm bunların dışında Windows server işletim sisteminde ve bazı sanallaştırma yazılımlarında **Nic Teaming** bulunmaktadır. Yani birden fazla etherneti tek bir ethernet gibi hem yedekli hem de yüksek bant genişlikli kullanma imkanı vardır.

Data Center Networks

Önceki başlığımızda bir fiziksel sunucunun içerisindeki birçok sanal sunucunun ağa nasıl bağlanması gerektiğinden bahsetmiştik. Peki yüzlerde fiziksel sunucu bulunan ve bunların da içerisinde binlerce sanal sistem bulunan bir Data Center'da ağ yapısı nasıldır?

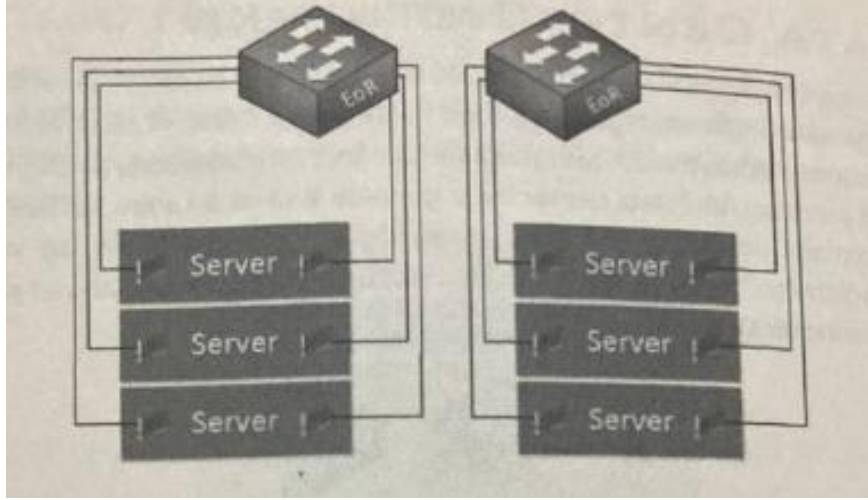
Data Center'larda genelde **2** ya da **4** kabinet olarak sistemler gruplanır ve bu sistemleri besleyen soğuk hava koridorları, ağ ve elektrik bağlantıları her bir grup için düşünülür. Genelde de her bir kabinet ya da grup üzerine bir **switch** konur.



Bu switch'e özel olarak **ToR** (Top of Rack) **switch** denir. Genelde de yedeklilik açısından ToR switch'ler **2 adet** olur. Böylece olası switch sorunlarında bağlantı kesintisiz olarak devam eder.

ToR switch'ler bir ara toplama switch'ine genelde de **fiber bağlantılar** ile taşınır. Bu bağlantılar genelde de 10 Gbps'lık hızlara sahiptir. Bu sistemin avantajı, her kabinetteki fiziksel sunuculardaki onlarca kablo, az sayıda fiber kablolarla toplama switchlerine aktarıldığından daha az kablo maliyeti olur ve sistem daha sadedir.

Bazen de birkaç rack kabindeki fiziksel sunucular direkt olarak ara switchlere bağlanıp, yine bu ara switchler bir **omurga switch**'te toplanabilir. Bu yapı daha az sayıda fakat daha yüksek portlu switch kullanımı gerektirir. Bu tür kullanılan switch'lere de **EoR** (End of Row) **switch**'ler denir.



Bu sistemin avantajı şüphesiz daha az sayıda switch olduğundan sorunun daha hızlı bulunabilmesi olabilir. Dezavantajı ise **artan kablo maliyetleridir**.

Cloud Computing

Bulut bilişim tüm süreçlerin otomatik olarak yürütüldüğü, satın alınan kiralanan hizmetlerin **otomatik hazırlandığı** bir sistemdir. Bunun dışında **NIST** (National Institute of Standards and Technology) bulut bilişimin özelliklerini aşağıdaki gibi sıralamaktadır:

- **On-demand self-service:** Ürün son kullanıcı tarafından otomatik olarak oluşturulabilmeli, hizmete girebilmeli ve istenildiği takdirde son kullanıcı tarafından sonlandırılabilmelidir.
- **Broad network access:** Son kullanıcı sunulan hizmete bilgisayar, tablet, mobil cihazlardan erişebilmelidir. Ayrıca istendiğinde son kullanıcı ile sunulan hizmet arası bağlantı vpn gibi güvenli yöntemleri desteklemelidir.
- **Resource pooling:** Bulut sağlayıcının bağlantısı dahil sunduğu hizmetler esnek yapıda olmalı. Gerektiğinde kaynaklar ve erişim hızı otomatik artırılabilmelidir.
- **Rapid elasticity:** Son kullanıcıya kaynaklar sınırsız görünmelidir. Son kullanıcı yalnızca ihtiyaçlarını belirtir, bulut bilişim tarafı ise bu kadar alan, disk, kapasite, hız yok dememelidir.
- **Measured service:** Bulut bilişim sağlayıcısı alınan hizmetlerin kullanımını şeffaf bir biçimde faturalandırmalıdır.

Service Model

Bulut bilişimde bizler bir fiziki ürün değil de bir **hizmet** satın alırız. Bazen satın aldığımız bu hizmeti satmak da isteyebiliriz. Bulut bilişim üzerinden sunulan ve sık olarak kullanılan **3 tür hizmet** vardır:

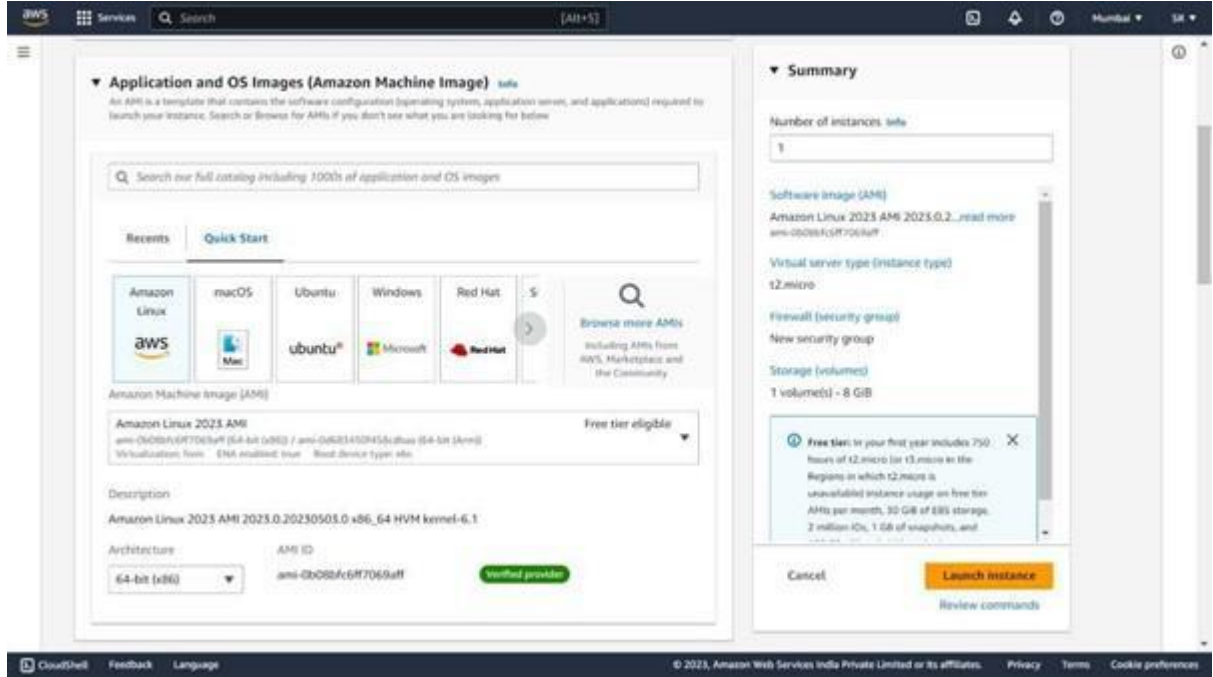
- **IaaS** - Infrastructure as a Service
- **PaaS** – Platform as a Service
- **SaaS** – Software as a Service

Şimdi bu hizmetleri daha yakından inceleyelim:

IaaS (Infrastructure as a Service)

IaaS sanal makinelere, router, switch, firewall, load balancer ve storage gibi Network sistemlerinin hizmet olarak sunulmasıdır.

Örnek olması açısından **AWS (Amazon Web Service)**'in sunduğu sanal sistemlere bakalım. Amazonda sadece sanal sisteminizin kaynaklarını seçmekle kalmaz, sanal cihazın işletim sistemini de kendiniz belirleyebilirsiniz.



Hangi işletim sistemini kullanacağınıza karar verdikten sonra işlemci, RAM, disk ve ağ bağlantısını seçebilirsiniz.

Birkaç dakikada istediğiniz sanal makineyi istediğiniz işletim sistemi ve istediğiniz özelliklerde hazır edebilir ve kullanmaya başlayabilirsiniz. Hiçbir manuel müdahale olmadan tüm süreç **otomatik** olarak işler. Bu tür sistemlerde de hizmet, **dakika/saat başına faturalandırılır**. Bu yüzden test için aldığınız ya da uzun süre kullanmayacağınız sistemleri kapatmanız bazı üreticilerde işinize yarayabilir. İsteddiğiniz zaman sanal sistemlerinizi de silebilirsiniz. Yine bu tür sanal sistemlerin **firewall, yedekleme, snapshot** gibi yan hizmetleri genelde ayrı olarak satılır. Bu özelliklerin tümü yalnızca Amazon'a özel değil, genelde **diğer bulut hizmetleri sunan platformlar da böyledir**.

PaaS (Platform as a Service)

Web tabanlı http/https üzerinden çalışan bir uygulama geliştirdiğinizi düşünelim. Bu durumda siz sunucu kiralama, sunucunun özellikleri, işletim sistemi ya da gerekli yazılımlarla zaman harcamak istemezsiniz. **Amazon AWS Beanstalk** ve **benzeri hizmetler** size hazır işletim sistemli, gerekli yazılımlar yüklenmiş, uygulamanızı çalıştırmaya, geliştirmeye hazır platformlar sunar.

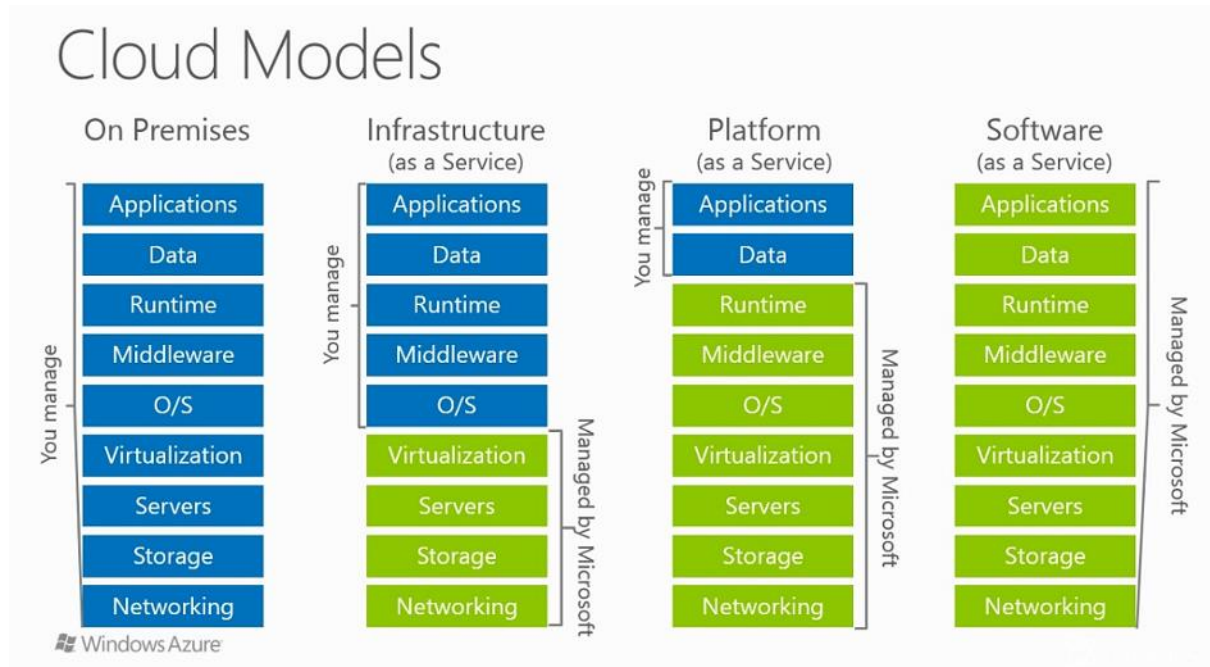
Bu tür platformlarda tek yapmanız gereken kendi geliştirdiğiniz yazılımı sisteme yükleyip, çalıştırmaktır. Benzer bir hizmeti **Google's App Engine** de sunmaktadır. Google App Engine de size yalnızca uygulamanızı upload edip çalıştırabileceğiniz bir **altyapı** sunar.

SaaS (Software as a Service)

SaaS (Software as a Service) bir son kullanıcının bir uygulama için abone olması ve hemen hiçbir şey yüklemeyen kullanabilmesine imkan sağlayan bir altyapıdır. Aslında bu altyapı adı size yeni gibi gelse de yıllardır kullandığımız birçok uygulama, bu altyapının ürünüdür:

- **Gmail**: Popüler e-mail sağlayıcısı
- **Microsoft Office 365**: Kaydolun ve online Office programlarını kullanın
- **Microsoft OneDrive**: Farklı cihazlardan buluttaki alanınıza rahatça erişebileceğiniz uygulama sunar

SaaS altyapısında bizler **sadece uygulamaya ücret** öderiz. Sunucu, bakım, yedekleme, güvenlik gibi ayrıntılarla uğraşmaz ve para ödemeyiz.



Public Cloud

Önceki başlıklarda anlatılan tüm Cloud hizmetleri büyük Cloud sağlayıcıları tarafından sağlanmaktadır ki biz bu tür genele açık Cloud hizmetlerine public Cloud diyoruz. Günümüzdeki en büyük Public Cloud sağlayıcıları:

- **Amazon AWS**
- **Microsoft Azure**
- **IBM**
- **Google Cloud Platform**

Bunlardan en yaygın güncel olarak Amazon AWS, diğerlerinden daha çok kullanılıyor durumda.

Private Cloud

Genelde tüm kullanıcıların, özellikle de ülkemizdeki resmi, özel kurumların tüm bilgilerini, veri tabanını kendi sahibi olmadığı bir Cloud sisteminde barındırması riskli olarak değerlendirilir. Hem cloud'taki verilerin güvenliği hem de verileri barındıran kuruluş tarafından kullanılması ihtimali haklı olarak korkutucu gelmektedir. Bu riskler, kuruluşları kendi **özel cloud**'unu kurmaya yöneltmiştir. Özellikle ülkemiz bu konuda ciddi adımlar atmış ve kamuya özel Cloud sistemi oluşturmuştur. Özelde ise şirketler kendi sunucularında, sistemlerinde hatta hiç yapamayan bile kendi NAS cihazında **free Cloud** yazılımları kullanarak kendi özel cloud'unu kullanmaktadır.

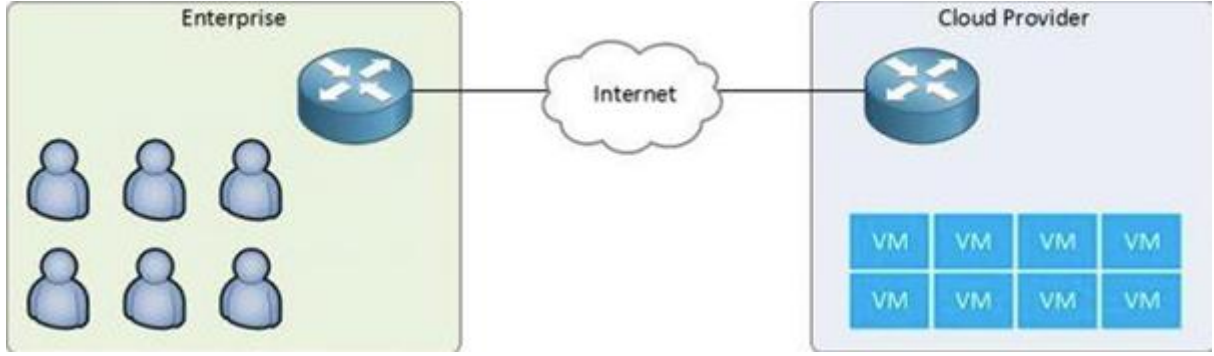
Kurum ve kuruluşlar kendi özel cloud'unu oluşturduğunda bu sistemin kendi içindeki yedekliliği, enerji ve erişim kesintilerine karşı önlemlerini, genişleme durumundaki stratejilerini, bu Cloud sistemine erişimlerdeki yetki ve ağ prosedürleri de kendileri yönetmek zorundadırlar. Sistem kurulduğunda artık Public Cloud'daki gibi IT birimleri ihtiyacı olan servislere birkaç adımda rahatça ulaşabilir olur.

Cloud Sistemlerine Güvenli Erişim

Cloud sistemlerinde ister Public isterse Private olsun bir de güvenli erişimi düşünüyor olmalıyız. En özel bilgilerimiz client ya da server sistemlerimizden kullandığımız cloud'a güvenli ve kesintisiz bir şekilde iletilmelidir.

Internet

Elbette **en çok kullanılan seçenektir**. Bulunduğumuz lokasyon ile Cloud sistemlerimiz arasındaki iletişimde interneti kullanmamızın bazı **avantaj** ve **dezavantajları** vardır. Önce avantajlarından başlayalım:



Üstteki şekildeki gibi bir yapımız olduğunu düşünelim. Tüm sanal sunucularımız Cloud tabanlı olup ve bu sunuculara internet aracılığıyla ulaşabiliyor olalım.

Cloud sistemlerimize **internet aracılığıyla ulaşmanın avantajları**:

- **Fiyat**: Cloud sistemlerimize internet ile erişmek özel private wan bağlantılarına kıyasla ciddi seviyede ekonomiktir.
- **Esneklik**: İnternet üzerinden erişimde alternatiflerimiz de çoktur. Hem her zaman her yerde birçok sağlayıcıyı bulmak hem de yedekli erişmek mümkündür.
- **Migration**: Cloud hizmetini aldığınız sağlayıcıyı değiştirdiğinizde tüm Cloud sağlayıcıları internet erişimini desteklediklerinden, bu değişimde bağlantı yönteminizi değiştirmeniz gerekmez.
- **Mobile kullanım**: Sahada ya da merkezde birçok mobil kullanıcınız var ise bu kullanıcıların internet erişimli mobil cihazlarından cloud'a erişmeleri de kolay olacaktır.

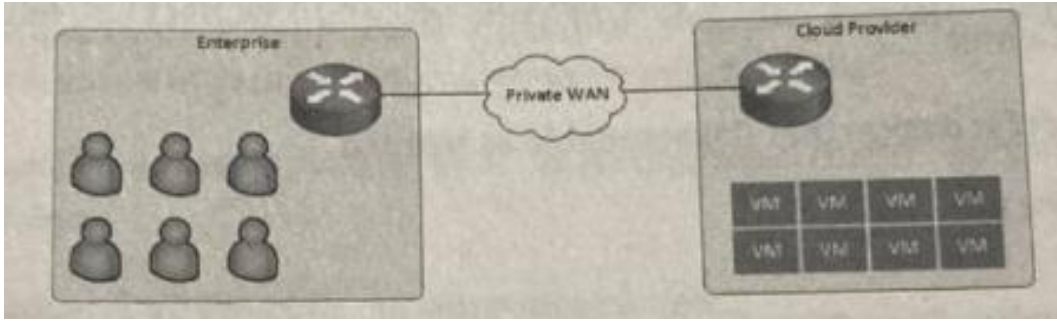
Cloud sistemlerimize **internet aracılığıyla ulaşmanın dezavantajları**:

- **Security**: İnternet erişimi olan her yerde güvenlik düşünülmesi gereken önemli bir aşamadır. Sonuçta hacker'lar man-in-the-middle atakları ile erişimdeki verileri ve şifreleri ele geçirebilmektedir.
- **Bandwidth**: Kritik bir yere internet erişimi ile ulaşabiliyorsanız karşıda çalıştırdığınız uygulamalara bağlı olarak erişmek isteyen kullanıcıya yeterli internet hızını sağlayabilmeniz gerekir.
- **QoS (Quality of Service)**: İnsanlar yalnızca Cloud erişimi için internet kullanmazlar. Anlık diğer ihtiyaçlar için gereken yüksek internet erişim hızları sizin Cloud bağlantınızı olumsuz etkileyebilir. Örneğin sizin kritik işlem yaptığınız esnada internetten film indiren, ya da dün akşam kaçırdığı diziyi bilgisayarından izleyen kullanıcılar sizin Cloud bağlantınızı olumsuz etkileyecektir. Bu durumda ağ geçidindeki cihazınızda (UTM/Firewall) QoS yapmanız ve cloud'a doğru olan trafiğe öncelik vermeniz gerekir. Bu da her zaman her koşulda, her cihazda kolay olmayabilir.
- **SLA (Service Level Agreements)**: Birçok internet sağlayıcısı size çok yüksek ücretler ödemediğiniz sürece kesintisiz bir erişim garantisinde bulunmazlar. Bu da internet yoksa Cloud erişimiz de yok demektir ve tüm işlerinizin durması

anlamına da gelebilir. Bu riske önlem olarak ya kesintisizlik anlaşması yapmalısınız ya da load balance çalışacak ve mümkünse farklı bir internet sağlayıcısının daha hizmetini kullanmalısınız.

Private WAN

İnternet ile erişim yöntemine **güvenli** bir alternatif de **private WAN** bağlantılarıdır. Bu hizmeti de yine sonuçta internet sağlayıcıları vermektedir fakat sizinle Cloud sağlayıcınız arasında **doğrudan** (İnternette geçmeyen, özel bağlantı) **erişim sağlanır**. Ülkemizde genelde internet erişim sağlayıcılarının da kendi bulut sistemleri vardır ve talep halinde özellikle kendi sistemleri üzerindeki Cloud sistemlerinize erişimlerde bu yöntem kullanılmaktadır.



Private WAN ile sistemlerimize ulaşmanın **avantajları**:

- **Bandwidth**: Private WAN teknolojileri internet, bağlantılarına kıyasla daha yüksek hız upload ve hız imkanları sunar.
- **SLA (Service Level Agreements)**: Bu tür bağlantılar zaten özel bir bağlantı olduğundan bağlantı kesintisizliği de genelde verilir.

Private WAN ile sistemlerimize ulaşmanın **dezavantajları**:

- **Cost**: Private WAN teknolojileri pahalıdır.
- **Availability**: Yeni bir şube ya da lokasyondan yeni bir bağlantı yeni bir yatırım gerektirir.
- **Flexibility**: Bu hizmeti sağlayan sağlayıcınızı kolay kolay değiştiremezsiniz. Size özel hat çekildiğinden farklı bir sağlayıcıya geçmek zordur. Zaten sağlayıcılar uzun yılları kapsayan sözleşme yaparlar.

Software Defined Network (SDN)

SDN (Yazılım Tabanlı Ağ), her yıl daha popüler hale gelen teknolojilerden biridir. Bazı ağ mühendisleri, tüm ağın gelecekte nasıl programlanacağına dair öyküler okur ya da duymuşlardır ve şimdi işlerinin C/C++, Java ya da Python'u bilen programcılar tarafından değiştirileceğinden korkuyorlar.

SDN nedir sorusunu cevaplamadan önce “geleneksel” ağa daha yakından bakmalıyız. Geleneksel ağın mevcut “sınırlamalarına” bakacağız, SDN'nin ne olduğunu ve SDN'in geleneksel ağın sahip olduğu “problemleri” nasıl çözeceğini açıklayalım.

Geleneksel Ağ

Belirli görevler için kullanılan yönlendiriciler, anahtarlar ve güvenlik duvarları gibi belirli ağ aygıtlarımız var.

Bu ağ aygıtları Cisco gibi ağ satıcıları tarafından satılır ve çoğu zaman özel donanım kullanılır. Yönlendiriciler için CCP (Cisco Configuration Protocol) ve Cisco ASA güvenlik duvarları için ASDM gibi bazı GUI ürünleri olmasına rağmen, bu aygıtların çoğu öncelikle CLI aracılığıyla yapılandırılır.

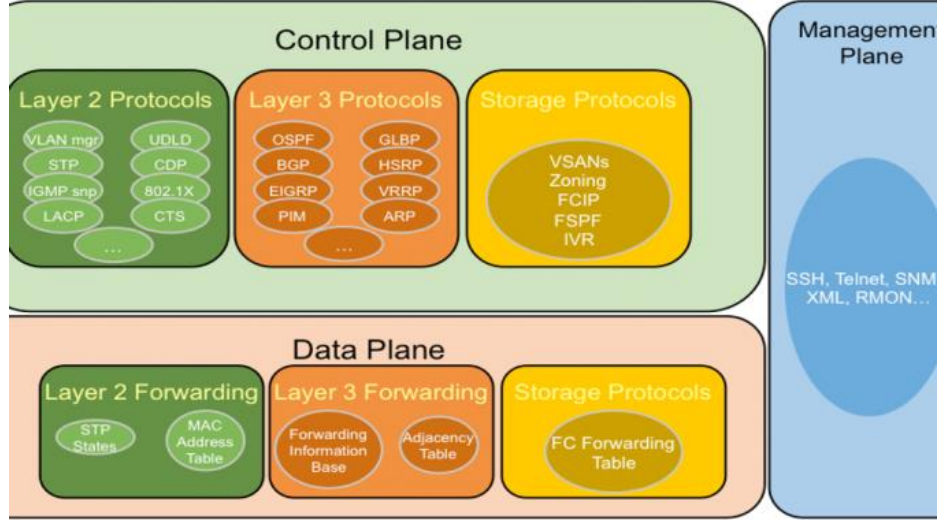
Bir ağ aygıtının, örneğin yönlendiricinin gerçekleştirmesi gereken farklı işlevleri vardır. Bir IP paketini iletme için yönlendiricinin yapması gereken bazı şeyler hakkında biraz düşünün:

- IP paketinin nereye yönlendirileceğini bulmak için yönlendirme tablosunda hedef IP adresini kontrol etmelidir.
- Yönlendirme tablosunda yüklü ağları öğrenmek için OSPF, EIGRP veya BGP gibi yönlendirme protokolleri gereklidir.
- Bir sonraki atlama veya hedefin hedef MAC adresini bulmak ve Ethernet çerçevesindeki hedef MAC adresini değiştirmek için ARP'yi kullanmak zorundadır.
- IP paketindeki TTL (Live Time), 1 ile azaltılmalı ve IP header checksum yeniden hesaplanmalıdır.

Bütün bu farklı görevler farklı düzlemlere ayrılır. Üç düzlem vardır:

- **Kontrol Düzlemi** (Control Plane)
- **Veri Düzlemi** (Data Plane)
- **Yönetim Düzlemi** (Management Plane)

Bu üç düzlem arasındaki farka bakalım..



Kontrol Düzlemi (Control Plane)

Kontrol düzlemi, yönlendirme bilgilerinin değiştirilmesi, ARP tablosunun oluşturulması vb. sorumludur. Kontrol düzlemi tarafından gerçekleştirilen bazı görevler şunlardır:

- Bir MAC adresi tablosu oluşturmak için MAC adreslerini öğrenmek.
- Döngü içermeyen bir topoloji oluşturmak için STP'yi çalıştırmak.
- ARP tabloları oluşturma.
- OSPF, EIGRP ve BGP gibi yönlendirme protokollerini çalıştırmak ve yönlendirme tablosunu oluşturmak.

Veri Düzlemi (Data Plane)

Veri akışı, trafiği iletmekten sorumludur. Kontrol düzleminin sağladığı bilgilere dayanır. Veri düzleminin ilgileneceği bazı görevler şunlardır:

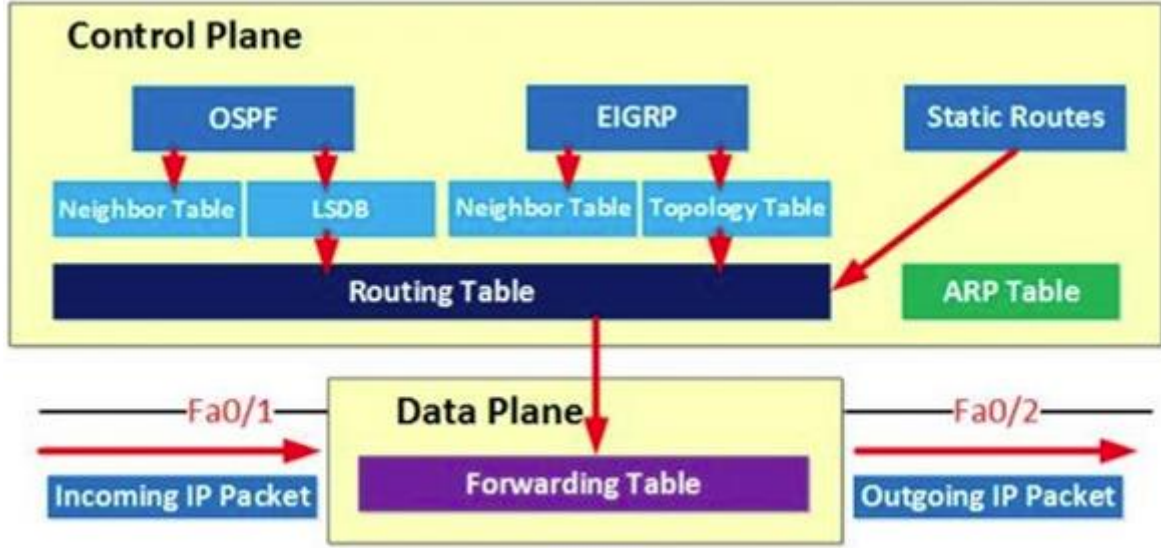
- Paketleri kapsüllemek ve açmak.
- 802.1Q başlığı gibi başlıkları ekleme veya çıkarma.
- Yönlendirme için MAC adreslerinin eşleştirilmesi.
- Yönlendirme tablosunda IP hedeflerini eşleme.
- NAT kullanırken kaynak ve hedef adresleri değiştirme.
- Erişim listeleri nedeniyle trafiği bırakma.

Veri düzleminin görevleri mümkün olduğunca hızlı bir şekilde gerçekleştirilmelidir, bu nedenle trafiğin iletilmesi ASIC'ler ve TCAM tabloları gibi donanımlar tarafından gerçekleştirilir.

Yönetim Düzlemi (Management Plane)

Yönetim düzlemi ağ cihazlarımızın erişimi ve yönetimi için kullanılır. Örneğin, cihazımıza telnet, SSH veya konsol portu üzerinden erişim.

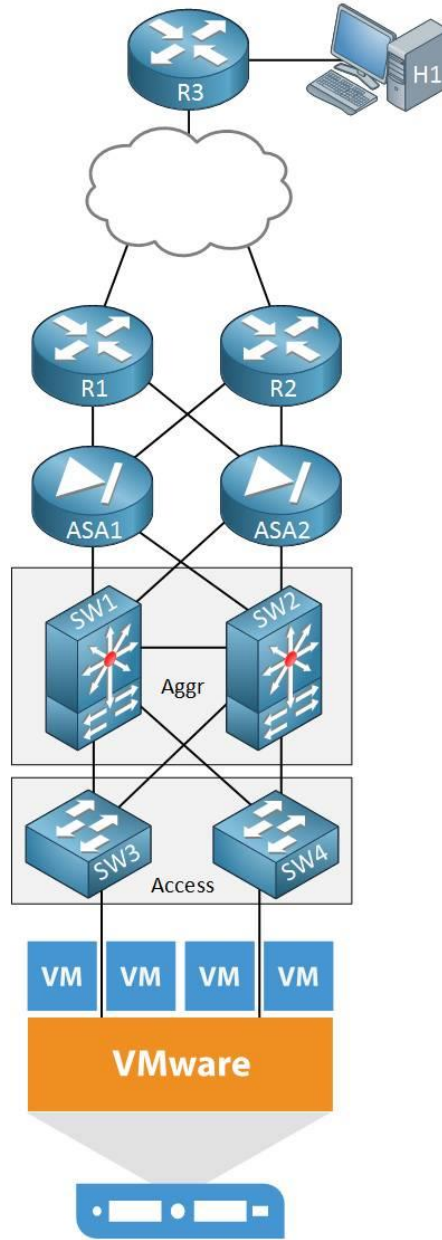
SDN tartışırken, kontrol ve veri düzlemi akılda tutulması gereken en önemli şeydir. Farklı düzlemleri görselleştirmenize yardımcı olacak kontrol ve veri düzleminin bir çizimi:



Yukarıda OSPF ve EIGRP gibi yönlendirme protokolleri ve bazı statik yönlendirme kullandığımız kontrol düzlemini görebilirsiniz. En iyi rotalar, yönlendirme tablosuna yüklenir. Yönlendiricinin yapması gereken bir başka tablo ARP tablosudur.

Yönlendirme tablosunu oluşturmak için yönlendirme ve ARP tablosundan bilgi kullanılır. Yönlendirici bir IP paketi aldığı anda, yönlendirme tablosu zaten oluşturulduğundan hızlı bir şekilde iletebilir.

Yukarıda anlatılan her şey, son 30 yıldır işleri yapmamızın yoluydu, bu yüzden geleneksel ağlarda bir yanlışlık yok. Bununla birlikte, günümüzde farklı çözümler isteyen bazı iş zorlukları var. Bir senaryoya bakalım.



Yukarıda bir şirketin veri merkezinin ağ altyapısını görüyoruz. Altta, bir dizi sanal makineye sahip bir VMware ESXi sunucusu buluyoruz. Bu sunucu, erişim ve toplama katmanlarından bazı anahtarlara bağlanır. Ayrıca sunucumuz ve iki yönlendiriciyi dış dünyaya erişim için koruyan iki ASA'yı da görüyoruz. Üstte, bir ana cihaz ile başka bir yönlendirici var.

Bu şirketin, VMware sunucusuna yüklenecek dört yeni sanal makine gerektiren yeni bir uygulama için bir iş gereksinimine sahip olduğunu varsayalım. Güvenlik nedenleriyle, her sanal makine farklı bir VLAN içinde olmalıdır. R3'ün arkasında H1 isimli bir kullanıcı, bu sanal makinelerde çalışan uygulamaya erişebilmelidir.

Bu yapmak için ağımda yapılandırmamız gereken bazı şeyleri düşünelim:

- VLAN'ların tüm anahtarlarda oluşturulması gerekir.
- Yeni VLAN'lar için bir kök köprüsü (STP) yapılandırmamız gerekiyor.
- Her bir VLAN için bir tane olmak üzere dört yeni alt ağ atamak zorundayız.
- Anahtarlarda IP adresleri ile yeni alt arayüzler oluşturmamız gerekiyor.
- Yeni VLAN'ların anahtarlarında VRRP veya HSRP'yi yapılandırmamız gerekiyor.
- Yeni uygulamalara / alt ağlara erişime izin vermek için güvenlik duvarlarını yapılandırmalıyız.
- Anahtarlarımız, yönlendiricilerimiz ve güvenlik duvarlarımızdaki bir yönlendirme protokolünde yeni alt ağların tanıtımını yapmak zorundayız.

Bize yardımcı olacak ağ otomasyon araçları olmasına rağmen, bu cihazların hepsini tek tek yapılandırmak için genellikle CLI kullanıyoruz. Bu yavaş, elle yapılan bir işlemdir ve bir insan yapmak zorundadır. Yeni bir sanal makineyi devreye almak sadece birkaç dakika sürerken, ağ ekibinin ağı hazırlaması birkaç saat sürebilir. Bunlar gibi değişiklikler de genellikle iş saatleri içinde değil, bakım saatleri sırasında yapılır.

Sunucu sanallaştırması, işletmelerin yukarıda açıklanan süreci hızlandıran bir şey aramasının nedenlerindendir. Sanallaştırmadan önce, tek bir işletim sistemine sahip tek bir fiziksel sunucumuz vardı. Günümüzde yüzlerce sanal makineye sahip birden fazla fiziksel sunucumuz var.

Bu sanal makineler otomatik olarak bir fiziksel sunucudan diğerine geçebilir. Bir L3 sınırını geçtiğinde, ağ ekibinin yönlendirme veya erişim listeleri için gerekli değişiklikleri yapmasını beklemek istemezsiniz ki tüm bunlar otomatik olmalıdır.

Günümüzde “trend”, her şeyin sanal olması gerektiğidir. Bunun ağlara da geldiğini görmek garip değil. Sadece tescilli donanım satmak için kullanılan Cisco gibi büyük şirketler artık VMware sunucularında çalıştırabileceğiniz sanal yönlendiriciler, ASA'lar, kablosuz LAN denetleyicileri vb. üretiyor.

Yazılım Tabanlı Ağ (SDN)

Birkaç yıl öncesine ait “bulut” kelimesi gibi, her kuruluş ya da satıcının, SDN'nin tam olarak ne olduğu ve sundukları farklı ürünler hakkında farklı bir fikri vardır.

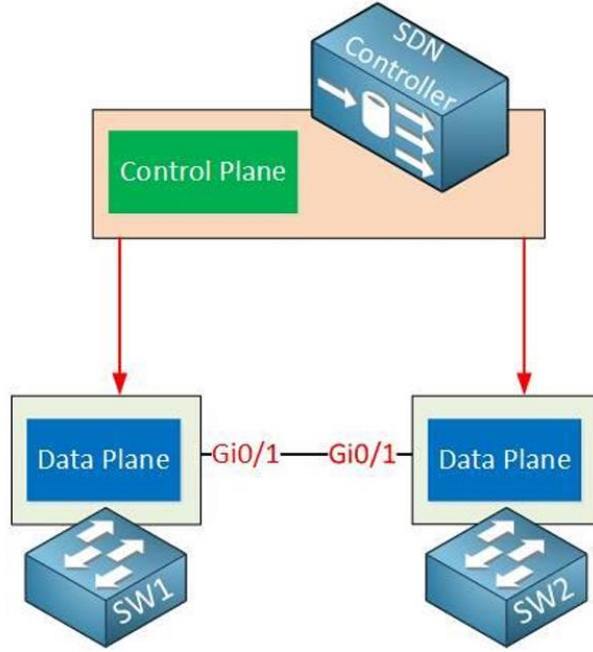
Geleneksel ağ, kontrol düzlemi için **dağıtılmış bir model** kullanır. ARP, STP, OSPF, EIGRP, BGP ve diğer protokoller her ağ cihazında ayrı ayrı çalışır. Bu ağ cihazları birbirleriyle iletişim kurar, ancak genel bakışa sahip veya tüm ağı kontrol eden merkezi bir cihaz yoktur.

Burada bir istisna (kablosuz ağa aşına olanlar için) kablosuz LAN denetleyicileridir (WLC). Kablosuz bir ağ yapılandırdığınızda, WLC'de erişim noktalarını kontrol eden

ve yapılandıran her şeyi yapılandırırınız. Artık her erişim noktasını ayrı ayrı yapılandırmamız gerekmiyor, hepsi WLC tarafından yapılıyor.

SDN ile **kontrol düzlemi için merkezi bir kontrolör** kullanıyoruz. Tedarikçinin SDN çözümüne bağlı olarak, bu, SDN kontrolörünün kontrol düzlemini %100 devraldığı veya sadece ağdaki tüm ağ cihazlarının kontrol düzleminde bir içgörüye sahip olduğu anlamına gelebilir. SDN denetleyici, fiziksel bir donanım aygıtı veya sanal bir makine olabilir.

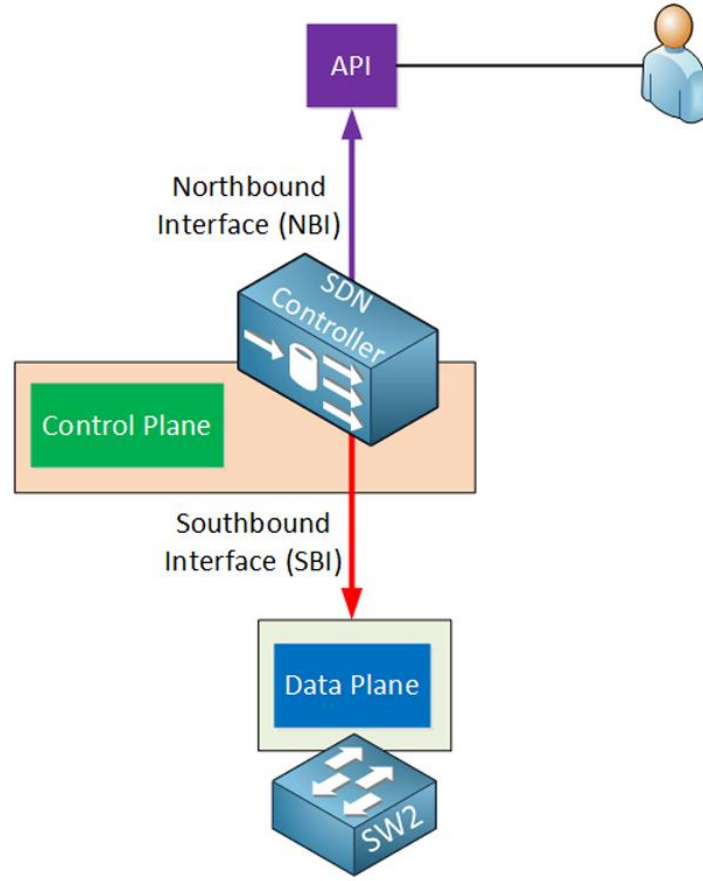
Bu görselleştirmemize yardımcı olacak bir örnek:



Yukarıda kontrol düzleminde sorumlu olan SDN kontrolcüsünü görebilirsiniz. Anahtarlar artık sadece **bir veri düzlemi olan, kontrol düzlemi olmayan** “aptal” cihazlardır. SDN kontrolörü, bu anahtarların veri düzlemini kontrol düzleminde gelen bilgilerle beslemekten sorumludur.

Dağıtılmış bir merkezi kontrol düzlemine sahip olmanın bazı avantajları ve dezavantajları vardır. Merkezi bir denetleyiciye sahip olmanın avantajlarından biri, tüm ağı tek bir aygıttan yapılandırabilmemizdir. Bu denetleyici, ağımda olan her şeye tam erişime ve içgörüye sahiptir.

Bu hikayeye biraz daha ayrıntı ekleyelim. SDN kontrolörü iki özel arayüz kullanır, aşağıdaki resme bir göz atın:



Arayüzler **kuzey yönlü arayüz (Northbound Interface- NBI)** ve **güney yönlü arayüz (Southbound Interface - SBI)** olarak adlandırılır. İkisini de açıklayalım:

Southbound Interface

SDN kontrolörü, veri düzlemini programlamak için ağ cihazlarımızla iletişim kurmalıdır. Bu güney yönlü arayüz üzerinden yapılır. Bu fiziksel bir arayüz değildir, bir yazılım arayüzü, genellikle bir API (Application Programming Interface – Uygulama Programlama Arayüzü) dir.

Bir API, önceden tanımlanmış işlevler ve veri yapıları kullanarak bir uygulamanın diğer uygulamalara erişimini sağlayan bir yazılım arabirimidir.

Bazı popüler güney yönlü arayüzler:

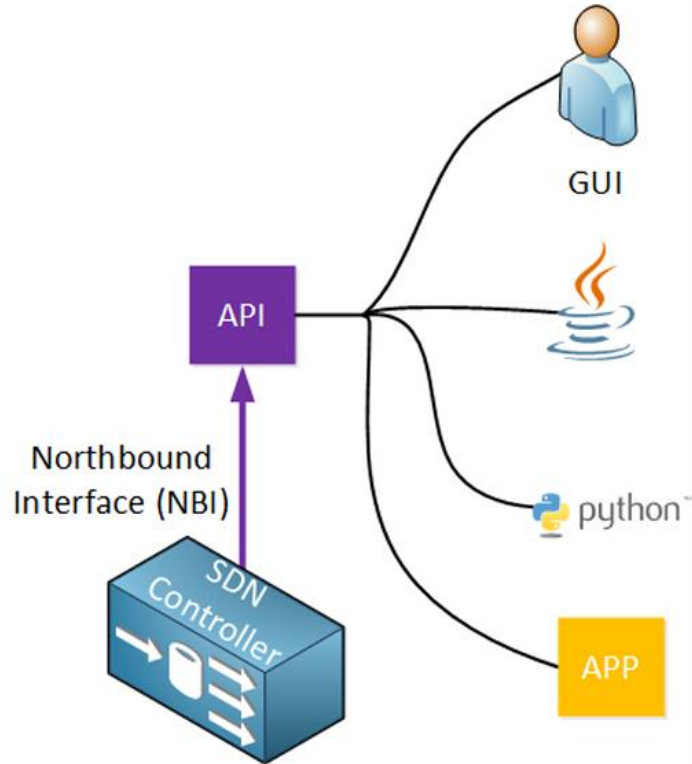
- **OpenFlow:** Bu muhtemelen şu an en popüler SBI'dır. Açık Ağ Vakfı tarafından sunulan açık kaynak kodlu bir protokoldür. OpenFlow'u destekleyen birkaç ağ aygıtı ve SDN denetleyicisi var.
- **Cisco OpFlex:** Bu, Cisco'nun OpenFlow'a verdiği cevabı Ayrıca standardizasyon için IETF'e sunulan açık kaynak kodlu bir protokoldür.
- **CLI:** Cisco, mevcut yönlendiriciler ve anahtarlar için bir SDN çözümü olan APIC-EM'yi sunuyor. Telnet, SSH ve SNMP gibi mevcut nesil donanımlarda bulunan protokolleri kullanır.

Northbound Interface

Northbound arayüzü, SDN kontrol ünitesine erişmek için kullanılır. Bu, bir ağ yöneticisinin onu yapılandırmak veya ondan bilgi almak için SDN'ya erişmesine izin verir. Bu bir GUI aracılığıyla yapılabilir, ancak aynı zamanda diğer uygulamaların SDN denetleyicisine erişmesine izin veren bir API sunar. Bunu komut dosyaları yazmak ve ağ yönetiminizi otomatikleştirmek için kullanabilirsiniz. İşte bazı örnekler:

- Ağınızdaki tüm ağ cihazlarındaki bilgileri listeleyin.
- Ağdaki tüm fiziksel arabirimlerin durumunu göster.
- Tüm anahtarlarınızda yeni bir VLAN ekleyin.
- Tüm ağınızın topolojisini göster.
- Yeni bir sanal makine oluşturulduğunda IP adreslerini, yönlendirme ve erişim listelerini otomatik olarak yapılandırın.

Bunu görselleştirmenize yardımcı olacak bir örnek:



API aracılığıyla, çok sayıda uygulama SDN denetleyicisine erişebilir:

- SDN denetleyiciden ağ hakkında bilgi almak için bir GUI kullanan bir kullanıcı, Sahne arakasında GUI API kullanıyor.

- Java ve Python'da yazılmış komut dosyaları, SDN denetleyicisinden bilgi almak veya ağı yapılandırmak için API'yi kullanabilir.
- Diğer uygulamalar SDN denetleyicisine erişebilir. Belki bir VMware ESXi sunucusunda yeni bir sanal makine oluşturulduğunda ağı otomatik olarak yapılandıran bir uygulama.

REST API

Birkaç kez, kuzey ve güneye bakan arayüzlerin API kullandığını belirttik. Bir API'nin ne olduğuna daha yakından bakalım. SDN denetleyicileri genellikle bir **REST API'sini (Representational State Transfer- Temsili Durum Transferi)** kullanır.

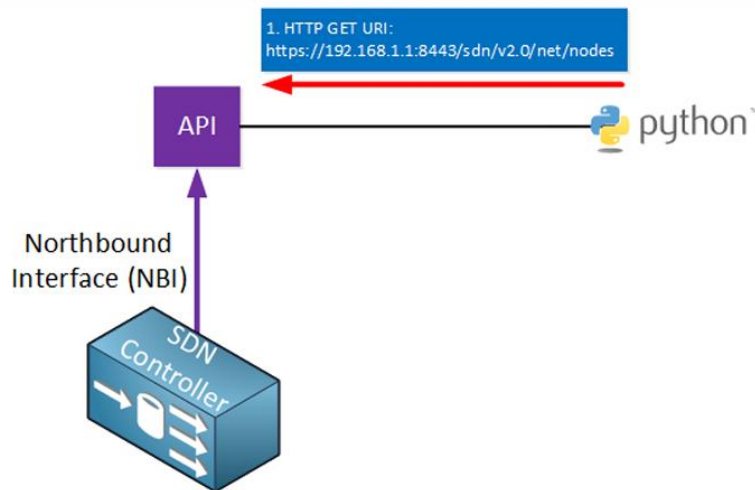
REST API, SDN denetleyicisi ve başka bir uygulama arasında bilgi göndermek ve almak için http mesajlarını kullanır. İnternette bir web sayfasına göz atarken veya çevrimiçi bir iletişim formu girdiğinizde kullandığınız http mesajlarını kullanır:

- http GET : bilgi almak istediğimizde kullanılır.
- http POST / PUT: bilgiyi yüklemek veya güncellemek istediğimizde kullanılır.

Bir web sayfasına göz atmakla benzerdir, sadece bu sefer bir web sayfası veya resim istemiyorsunuz, ancak SDN denetleyicisinden belirli bir nesne, örneğin, ağdaki tüm VLAN'ları içeren bir liste istiyorsunuz.

SDN denetleyicisi, http GET isteğini aldığı anda, talep edilen bilgilerle bir http GET yanıtı ile yanıt verecektir. Bu bilgi ortak bir veri formatında teslim edilir. En çok kullanılan iki veri formatı:

- JSON (JavaScript Object Notation – JavaScript Nesne Gösterimi)
- XML (eXtensible Markup Language)

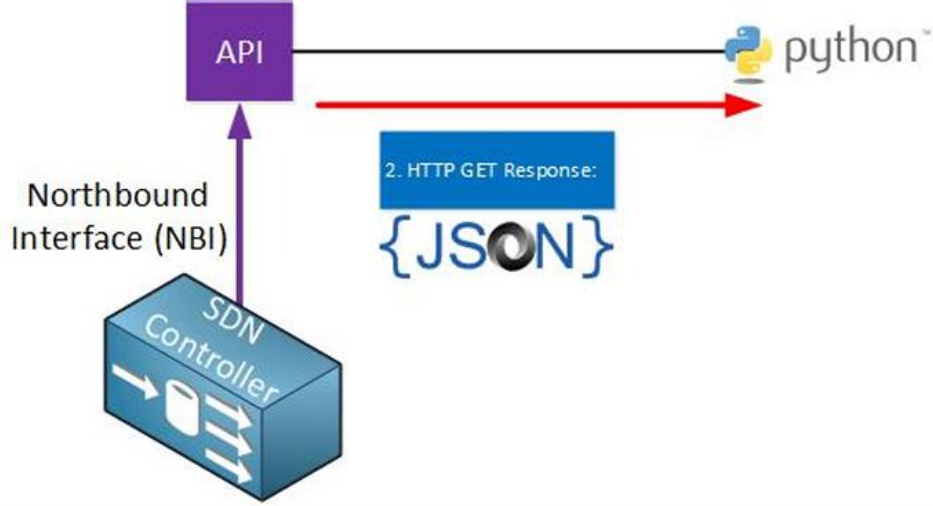


Yukarıda, aşağıdaki URL'yi API aracılığıyla almak için http GET kullanan bir Python betiğimiz var:

<https://192.168.1.1:8443/sdn/v2.0/net/nodes>

Bu URL, ağıdaki tüm düğümler (ana makineler) hakkındaki bilgileri gibi bazı değişkenleri alacaktır.

API bunu aldıktan sonra bir http GET yanıt mesajı ile yanıt verir:



İstenen değişkenler JSON formatında sağlanacaktır ve işte böyle görünüyor:

```
{
  "nodes": [
    {
      "ip": "172.16.1.1",
      "mac": "fa16.3e5d.f1f4",
      "vid": 0,
      "dpid": "00:00:00:00:00:00:00:03",
      "port": 1
    }, {
      "ip": "172.16.1.2",
      "mac": "fa16.3e5d.f1f5",
      "vid": 0,
      "dpid": "00:00:00:00:00:00:00:03",
      "port": 2
    }
  ]
}
```

Daha önce hiç JSON görmemiş olsanız bile, yukarıdaki çıktının okunması kolaydır. Bu ağda iki düğümün bulunduğunu söylüyor ve IP adreslerini ve MAC adreslerini gösteriyor.

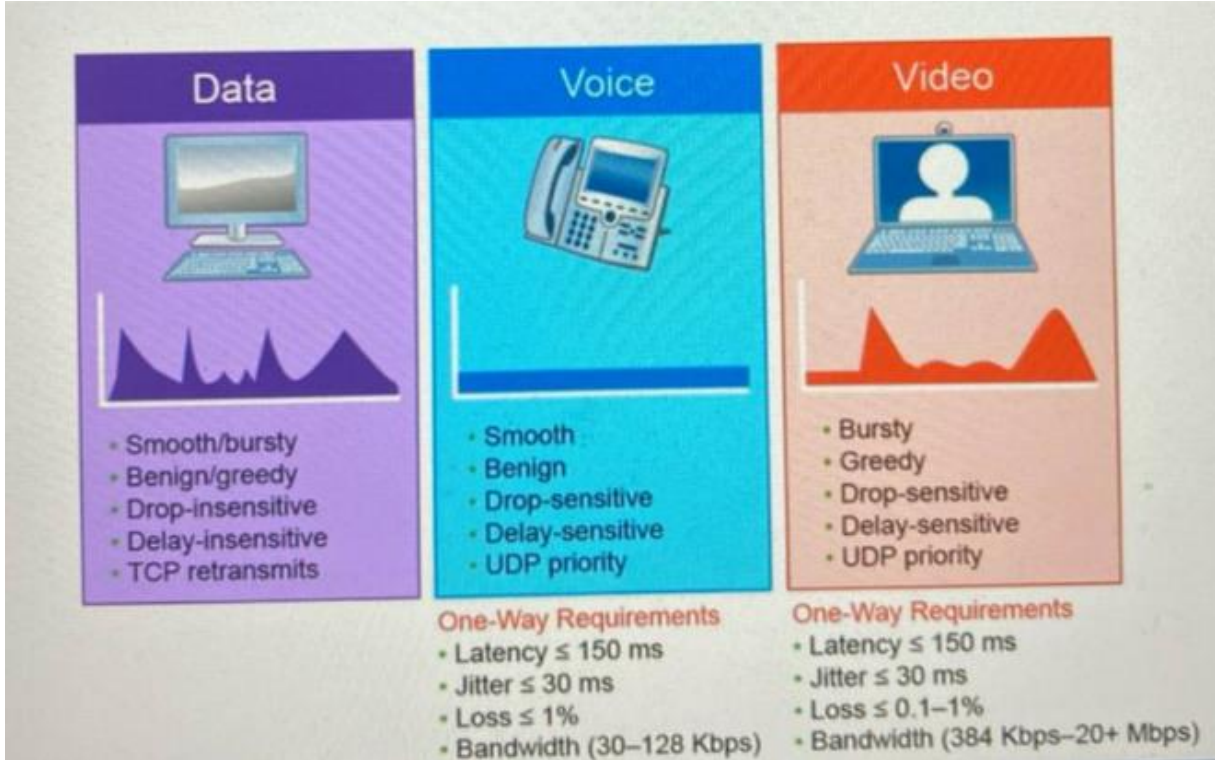
Servis Kalitesi (QoS)

QoS (Quality of Service), bir ağıdaki farklı türdeki trafiklerin farklı önceliklerle işlenmesini sağlayan mekanizmaların tümüdür. Amaç tüm trafiğe eşit davranmak yerine:

- Ses (VoIP) → düşük gecikme
- Video konferans → kesintisiz akış
- Finansal uygulamalar → kritik öncelik
- Normal veri trafiği → daha düşük öncelik

Gibi ihtiyaçlara göre paketlerin önceliklendirilmesi (priority), sıralanması (queuing) ve yönetilmesidir.

QoS özellikle tıkalı (congested) ağlarda önem kazanır. Bant genişliği yeterliyse QoS'e gerek yoktur; mekanizmanın amacı kıt kaynaklarda önemli trafiği kurtarmaktır.

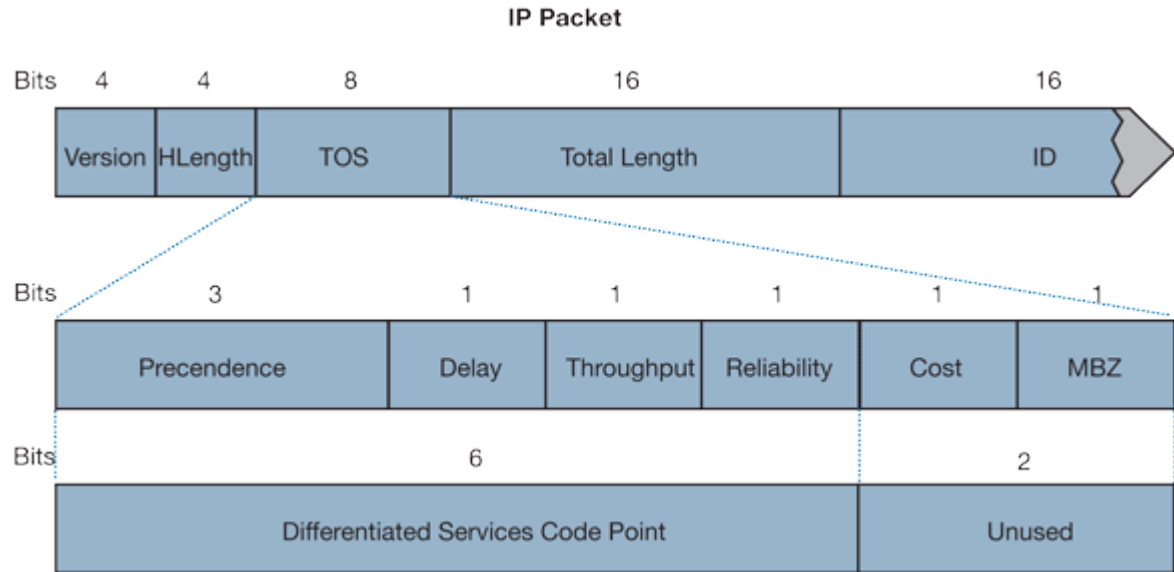


QoS Sınıflandırma (Classification) ve İşaretleme (Marking)

QoS Classification, trafiğin hangi sınıfa ait olduğunu belirleme işlemidir.

Router/Switch gelen pakete bakar ve “bu VoIP, bu video, bu SSH bu bulk data” gibi karar verir. Temel amaç, her trafik türünü farklı bir “class” içine almak ve ileride farklı işleme tabi tutmaktır.

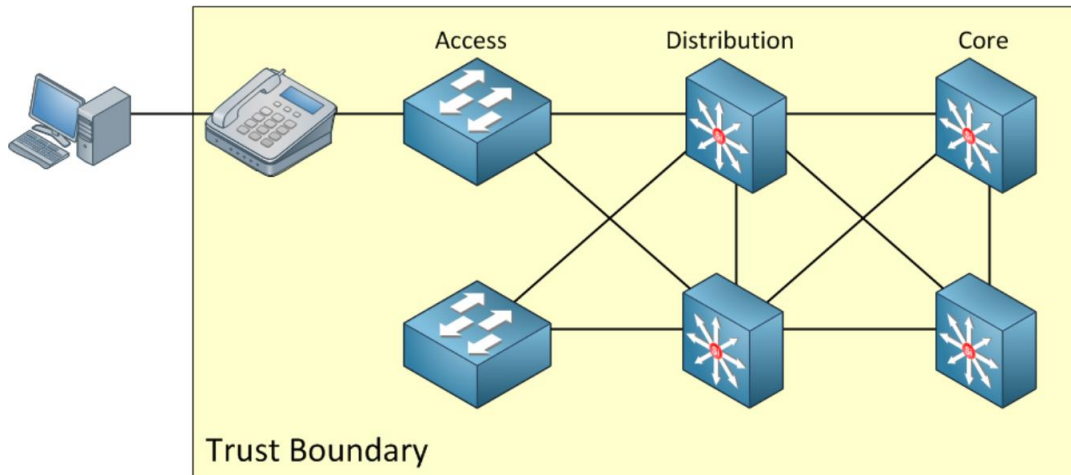
QoS Marking, sınıflandırılan trafiğe bir etiket (mark) koyma işlemidir. Bu etiket sayesinde ağın geri kalan cihazları paketin önem seviyesini bilir.



İşaretleme türlerinden DSCP (Differentiated Services Code Point) – Layer 3, IP header içindeki 6 bitlik alan. Örnek olarak , EF (Expedited Forwarding - VoIP) → 46, AF41/AF42/AF43 (Video), CS50(Best-effort – normal trafik) gibi değerler alabilir.

QoS Trust Boundaries

QoS işaretlemelelerinde önemli konulardan biri Trust Boundary yani Güven Sınırı'dır.

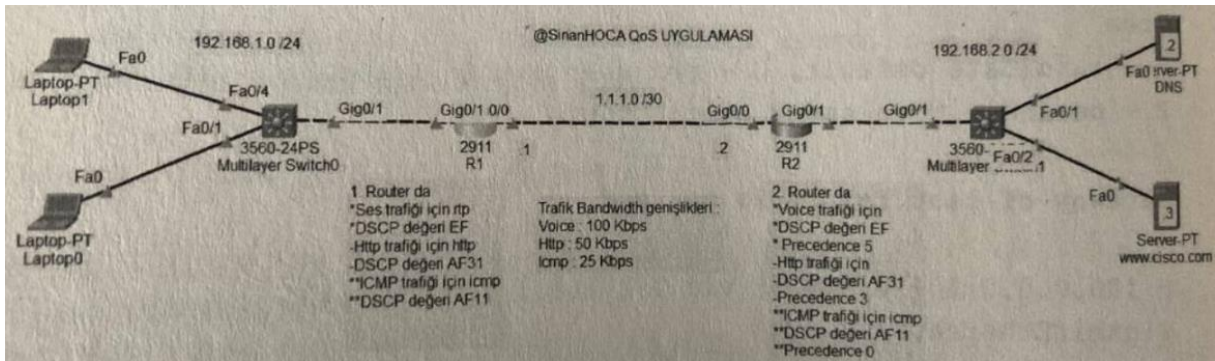


Bir uç kullanıcı (PC) kendi trafiğini DSCP = EF (46) yapıp VoIP gibi işaretleyebilir. Eğer switch buna inanırsa, tüm trafik en yüksek önceliği alır ve QoS çöker. Bu nedenle kurum ağlarında QoS'ta her paketin işaretine güvenilmez.

QoS Yapılandırma

QoS, temel olarak bizim belirleyeceğimiz belli özellikteki ya da belli IP bloklarının Switch ve Router'da önceliklendirilmesidir. Aynı Switch'e bağlı IP telefonlar, IP kamera ve personellerin olduğunu düşünün. Öncelikle Switch'lerin de bir **performans** sınırı vardır, bu genel olarak toplam Gigabit Per Second cinsinden toplam değeridir. Örneğin 24 portlu standart bir Switch'in **150-400 Gbps** civarındadır. Bu değeri bu Switch'in, aynı anda en fazla bu kadar trafiği portlar arası iletebilecek olarak düşünebiliriz.

Benzer kısıtlar Router'lar için de vardır. Aynı Switch'e bağlı bir personelin çok yoğun trafik geçirmesi durumunda bu Switch'teki bağlı IP telefonların paketlerinde gecikmeler olabilir ve bu da seste yankılanma hatta kesintilere yol açabilir. Dolayısıyla bu Switch için ses paketlerini önceliklendirmek, hatta gerektiğinde diğer trafik türlerinin yavaşlatılması ama ses trafiğinin hep hızlı ve öncelikli olmasını sağlamalıyız. İşte bu önceliklendirmeleri QoS ile yapıyoruz. İnsanlar QoS karışık olduğundan ve bu tür risklere girmek istemediğinden doğrudan donanımsal olarak Switch ve network olarak IP telefon, Data, IP kamera networklerini ayırabilirler. Yani bu yoğun farklı trafikler için farklı Switch'ler kullanılıyor genelde. Konuyu bir örnek ile uygulamalı olarak görelim:



Yukarıdaki uygulamada standart ayarlamaları yaptığımızı ve soldaki kullanıcıların sağdaki sunuculara ulaşabildiğini düşünelim. Şimdi soldan sağa doğru akan trafikte **ses**, **http** ve **icmp** paketlerini önceliklendirelim. Aslına bakarsak önceliklendirmeye Switch'lerde bile başlayabiliriz. Genel olarak QoS'un uygulamalarında şu aşamalar bulunmaktadır.

1. Önceliklendirilecek trafiklerin belirlenmesi için **Class-Map** tanımlanmalıdır. Eğer bir trafik türünü değil de bir IP bloğunu önceliklendireceksek (Ör Vip-Personel IP bloğu) o zaman önce bu IP bloğu için **ACL** yazarız sonra bu ACL'i Class-Map'de tanımlayabiliriz.
2. Oluşturduğumuz Class-Map'leri **Policy-Map**'de birleştiriyoruz.
3. Tanımladığımız Policy-Map'i trafiğinin geliş ya da gidiş yönüne göre ilgili **router** ya da **switch** interface'de tanımlanması gerekir.

Genel olarak AF değerlerinin karşılaştırma tablosu:

Drop	Class 1	Class 2	Class 3	Class 4
Low	001010 AF11	010010 AF21	011010 AF31	100010 AF41
Medium	001100 AF12	010100 AF22	011100 AF32	100100 AF42
High	001110 AF13	010110 AF23	011110 AF33	100110 AF43

Şimdi 1. Router'da önceliklendireceğimiz trafikleri class-map ile tanımlayalım:

Router'dan geçen rtp paketlerini **voice** başlığı altında tanımlayalım:

```
R1(config)#class-map voice
R1(config-cmap)#match protocol rtp
R1(config-cmap)#exit
```

Router'dan geçen http paketlerini **http** başlığı altında tanımlayalım:

```
R1(config)#class-map http
R1(config-cmap)#match protocol http
R1(config-cmap)#exit
```

Router'dan geçen icmp paketlerini **icmp** başlığı altında tanımlayalım:

```
R1(config)#class-map icmp
R1(config-cmap)#match protocol icmp
R1(config-cmap)#exit
```

Şimdi de tanımladığımız trafiklerin önceliklerini ve bant genişliklerini belirleyelim:

```
R1(config)#policy-map sinan
R1(config-pmap)#class voice <<<< buradaki isimler class-map de verdiğimiz isim olmalı
R1(config-pmap-c)#set ip dscp ef
R1(config-pmap-c)#priority 100
R1(config-pmap-c)#exit
R1(config-pmap)#class http <<<< buradaki isimler class-map de verdiğimiz isim olmalı
R1(config-pmap-c)#set ip dscp af31
R1(config-pmap-c)#bandwidth 50
R1(config-pmap-c)#
R1(config-pmap-c)#exit
R1(config-pmap)#class icmp <<<< buradaki isimler class-map de verdiğimiz isim olmalı
R1(config-pmap-c)#set ip dscp af11
R1(config-pmap-c)#bandwidth 25
R1(config-pmap-c)#exit
```

Şimdi de tanımladığımız Policy-map'i gidiş yönündeki **interface**'de devreye alalım:

```
R1(config)#int gi 0/0
R1(config-if)#service-policy output sinan
R1(config-if)#exit
```

Şimdi de 2. Router'da ayarlamaları yapalım. QoS ayarları kaynak ile hedef arasındaki tüm Router'larda uygulanmalıdır.

```
R2(config)#class-map voice
R2(config-cmap)#match ip dscp ef
R2(config-cmap)#exit
R2(config)#class-map http
R2(config-cmap)#match ip dscp af31
R2(config-cmap)#exit
R2(config)#class-map icmp
R2(config-cmap)#match ip dscp af11
R2(config-cmap)#exit

Şimdi de policy-map'ler tanımlayalım:

R2(config)#policy-map sinan2
```

```
R2(config-pmap)#class voice
R2(config-pmap-c)#set precedence 5
R2(config-pmap-c)#exit
R2(config-pmap)#class http
R2(config-pmap-c)#set precedence 3
R2(config-pmap-c)#exit
R2(config-pmap)#class icmp
R2(config-pmap-c)#set precedence 0
R2(config-pmap-c)#exit
```

Şimdi de bu Router'a giriş interface'inde giriş yönünde bu policy-map'i tanımlayalım:

```
R2(config-pmap)#int gi 0/0
R2(config-if)#service-policy input sinan2
R2(config-if)#end
```

Şimdi yaptığımız **ayarlara** ve **test öncesi değerlere** bakalım:

```
R1#sh policy-map
Policy Map sinan
Class voice
  Strict Priority
  Bandwidth 100 (kbps) Burst 2500 (Bytes)
  set ip dscp ef
Class http
  Bandwidth 50 (kbps) Max Threshold 64 (packets)
  set ip dscp af31
Class icmp
  Bandwidth 25 (kbps) Max Threshold 64 (packets)
  set ip dscp af11

R1#sh policy-map int gi 0/0
GigabitEthernet0/0

Service-policy output: sinan

Class-map: voice (match-all)
  0 packets, 0 bytes
  5 minute offered rate 0 bps, drop rate 0 bps
  Match: protocol rtp
```

```
QoS Set
  dscp ef
  Packets marked 0
Queueing
  Strict Priority
  Output Queue: Conversation 264
  Bandwidth 100 (kbps) Burst 2500 (Bytes)
  (pkts matched/bytes matched) 0/0
  (total drops/bytes drops) 0/0

Class-map: http (match-all)
  0 packets, 0 bytes
  5 minute offered rate 0 bps, drop rate 0 bps
  Match: protocol http
  QoS Set
    dscp af31
    Packets marked 0
  Queueing
    Output Queue: Conversation 265
    Bandwidth 50 (kbps)Max Threshold 64 (packets)
    (pkts matched/bytes matched) 0/0
    (depth/total drops/no-buffer drops) 0/0/0

Class-map: icmp (match-all)
  0 packets, 0 bytes
  5 minute offered rate 0 bps, drop rate 0 bps
  Match: protocol icmp
  QoS Set
    dscp af11
    Packets marked 0
  Queueing
    Output Queue: Conversation 266
    Bandwidth 25 (kbps)Max Threshold 64 (packets)
    (pkts matched/bytes matched) 0/0
    (depth/total drops/no-buffer drops) 0/0/0

Class-map: class-default (match-any)
  145 packets, 19684 bytes
  5 minute offered rate 63 bps, drop rate 0 bps
  Match: any
```

Şimdi de Client'dan web sitesine girelim ve sonrasında değerleri tekrar kontrol edelim:

```

R1#sh policy-map int gi 0/0
GigabitEthernet0/0

Service-policy output: sinan

Class-map: voice (match-all)
  0 packets, 0 bytes
  5 minute offered rate 0 bps, drop rate 0 bps
  Match: protocol rtp
  QoS Set
    dscp ef
    Packets marked 0
  Queueing
    Strict Priority
    Output Queue: Conversation 264
    Bandwidth 100 (kbps) Burst 2500 (Bytes)
    (pkts matched/bytes matched) 0/0
    (total drops/bytes drops) 0/0

Class-map: http (match-all)
  8 packets, 333 bytes
  5 minute offered rate 15 bps, drop rate 0 bps
  Match: protocol http
  QoS Set
    dscp af31
    Packets marked 8
  Queueing
    Output Queue: Conversation 265
    Bandwidth 50 (kbps)Max Threshold 64 (packets)
    (pkts matched/bytes matched) 0/0
    (depth/total drops/no-buffer drops) 0/0/0

```

Aynı şekilde 2. Router’ımızda da paketlerin önceliklendirildiğini görebiliriz:

```

R2#sh policy-map int gi 0/0
GigabitEthernet0/0

Service-policy input: sinan2

Class-map: voice (match-all)
  0 packets, 0 bytes
  5 minute offered rate 0 bps, drop rate 0 bps
  Match: ip dscp ef (46)
  QoS Set
    precedence 5
    Packets marked 0

Class-map: http (match-all)
  8 packets, 333 bytes
  5 minute offered rate 15 bps, drop rate 0 bps
  Match: ip dscp af31 (26)
  QoS Set
    precedence 3
    Packets marked 8

```